

C++ Inheritance

C++ Inheritance

In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.

In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

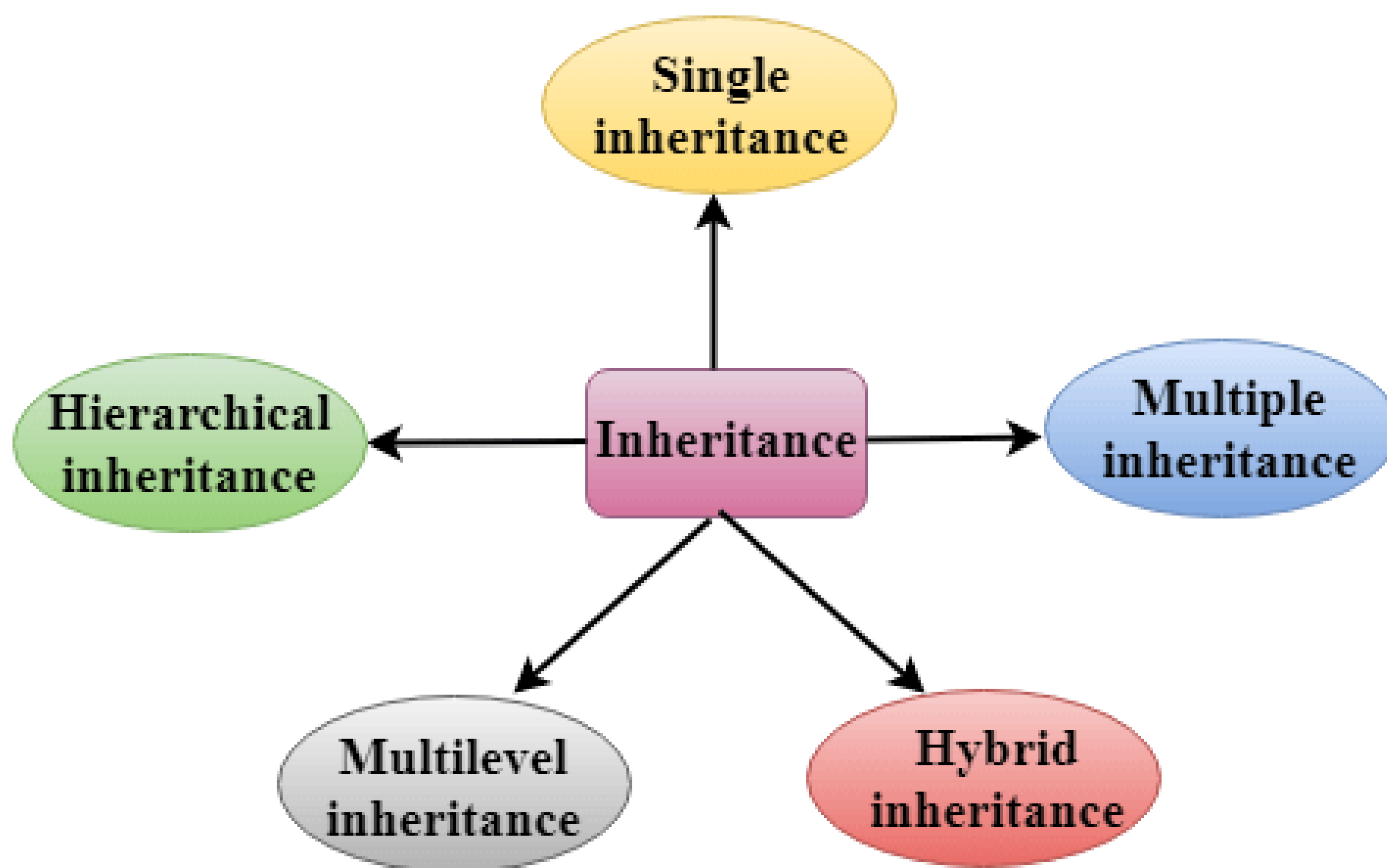
Advantage of C++ Inheritance

Codereusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Types Of Inheritance

C++ supports five types of inheritance:

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance



Derived Classes

A Derived class is defined as the class derived from the base class.

The Syntax of Derived class:

```
class derived_class_name :: visibility-mode base_class_name  
  
{  
  
    // body of the derived class.  
  
}
```

Where,

derived_class_name: It is the name of the derived class.

visibility mode: The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.

base_class_name: It is the name of the base class.

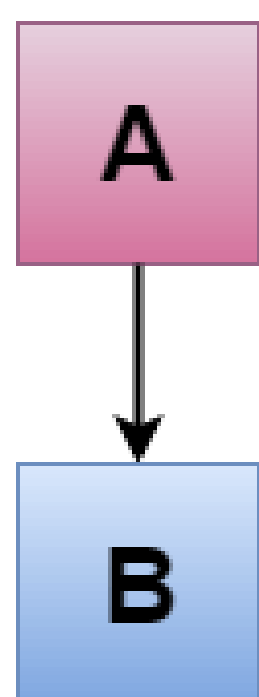
- When the base class is privately inherited by the derived class, public members of the base class becomes the private members of the derived class. Therefore, the public members of the base class are not accessible by the objects of the derived class only by the member functions of the derived class.
- When the base class is publicly inherited by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are accessible by the objects of the derived class as well as by the member functions of the base class.

Note:

- In C++, the default mode of visibility is private.
- The private members of the base class are never inherited.

C++ Single Inheritance

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

C++ Single Level Inheritance Example: Inheriting Fields

When one class inherits another class, it is known as single level inheritance. Let's see the example of single level inheritance which inherits the fields only.

```
#include <iostream>

using namespace std;

class Account {

public:

    float salary = 60000;

};

class Programmer: public Account {

public:

    float bonus = 5000;

};

int main(void) {

    Programmer p1;

    cout<<"Salary: "<<p1.salary<<endl;

    cout<<"Bonus: "<<p1.bonus<<endl;

    return 0;
```



```
}
```

Output:

```
Salary: 60000
```

```
Bonus: 5000
```

In the above example, Employee is the **base** class and Programmer is the **derived** class.

C++ Single Level Inheritance Example: Inheriting Methods

Let's see another example of inheritance in C++ which inherits methods only.

```
#include <iostream>
```

```
using namespace std;
```

```
class Animal {
```

```
public:
```

```
void eat() {
```

```
    cout<<"Eating..."<<endl;
```

```
}
```

```
};
```

```
class Dog: public Animal
```

```
{
```

```
public:
```

```

    void bark() {

        cout<<"Barking...";

    }

};

int main(void) {

    Dog d1;

    d1.eat( );

    d1.bark( );

    return 0;

}

```

Output:

```

Eating...

Barking...

```

Let's see a simple example.

```

#include <iostream>

using namespace std;

class A

{

```

```

int a = 4;

int b = 5;

public:

int mul()

{

    int c = a*b;

    return c;

}

};

class B : private A

{

public:

void display()

{

    int result = mul();

    std::cout <<"Multiplication of a and b is : "<<result<< std::endl;

}

```



```
};

int main()

{

    B b;

    b.display();


    return 0;

}
```

Output:

Multiplication of a and b is : 20

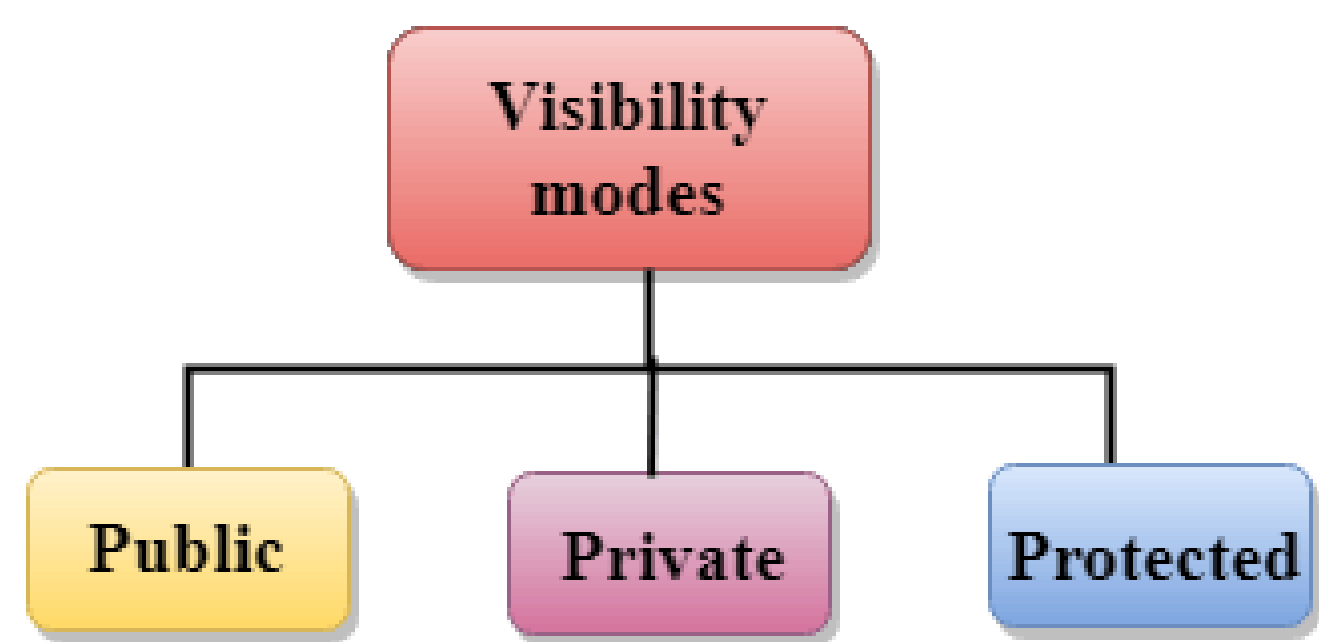
In the above example, class *A* is privately inherited. Therefore, the `mul()` function of class '*A*' cannot be accessed by the object of class *B*. It can only be accessed by the member function of class *B*.

How to make a Private Member Inheritable

The private member is not inheritable. If we modify the visibility mode by making it public, but this takes away the advantage of data hiding.

C++ introduces a third visibility modifier, i.e., **protected**. The member which is declared as protected will be accessible to all the member functions within the class as well as the class immediately derived from it.

Visibility modes can be classified into three categories:



- **Public:** When the member is declared as public, it is accessible to all the functions of the program.
- **Private:** When the member is declared as private, it is accessible within the class only.
- **Protected:** When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

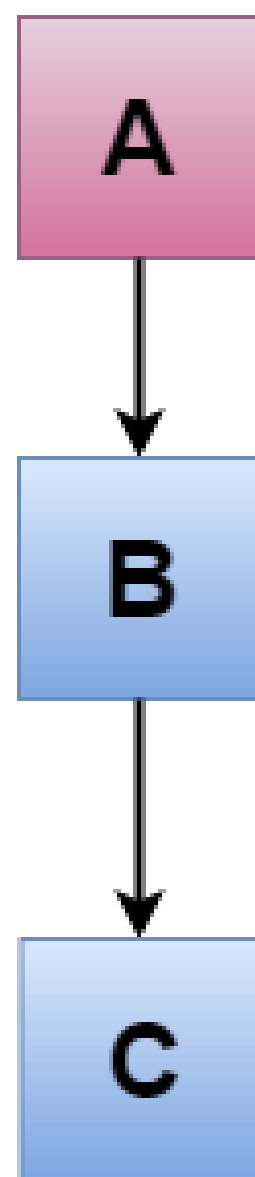
Visibility of Inherited Members

Base class visibility	Derived class visibility		
	Public	Private	Protected
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected

Public	Public	Private	Protected
--------	--------	---------	-----------

C++ Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class.



C++ Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

Let's see the example of multi level inheritance in C++.

```
#include <iostream>
```

```
using namespace std;
```



```

class Animal {

    public:

    void eat() {

        cout<<"Eating..."<<endl;

    }

};

class Dog: public Animal

{

    public:

    void bark(){

        cout<<"Barking..."<<endl;

    }

};

class BabyDog: public Dog

{

    public:

    void weep() {

        cout<<"Weeping...";

```

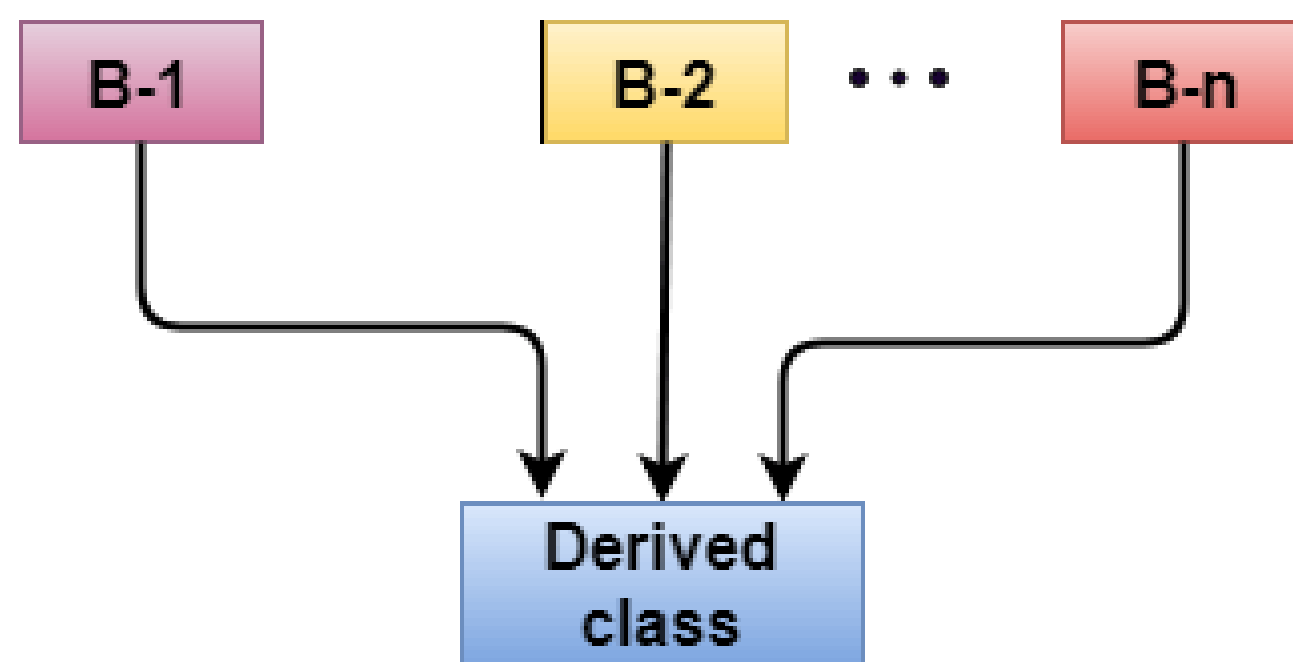
```
    }  
  
};  
  
int main(void) {  
  
    BabyDog d1;  
  
    d1.eat( );  
  
    d1.bark( );  
  
    d1.weep( );  
  
    return 0;  
  
}
```

Output:

```
Eating...  
  
Barking...  
  
Weeping...
```

C++ Multiple Inheritance

Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.



Syntax of the Derived class:

```

class D : visibility B-1, visibility B-2, ?
{
    // Body of the class;
}
  
```

Let's see a simple example of multiple inheritance.

```

#include <iostream>

using namespace std;

class A
{
    protected:

    int a;

    public:

    void get_a(int n)
  
```

```

    {

        a = n;

    }

};

class B

{

    protected:

        int b;

    public:

        void get_b(int n)

        {

            b = n;

        }

};

class C : public A, public B

{

    public:

        void display()

```

```

{

    std::cout << "The value of a is : " <<a<< std::endl;

    std::cout << "The value of b is : " <<b<< std::endl;

    cout<<"Addition of a and b is : "<<a+b;

}

};

int main()

{

    C c;

    c.get_a(10);

    c.get_b(20);

    c.display();

    return 0;

}

```

Output:

The value of a is : 10

The value of b is : 20

Addition of a and b is : 30

In the above example, class 'C' inherits two base classes 'A' and 'B' in a public mode.

Ambiguity Resolution in Inheritance

Ambiguity can be occurred in using the multiple inheritance when a function with the same name occurs in more than one base class.

Let's understand this through an example:

```
#include <iostream>

using namespace std;

class A
{
    public:

    void display()
    {
        std::cout << "Class A" << std::endl;
    }
};

class B
{
    public:
```

```

void display()

{

    std::cout << "Class B" << std::endl;

}

};

class C : public A, public B

{

    void view()

    {

        display();

    }

};

int main()

{

    C c;

    c.display();

    return 0;

}

```

Output:

```
error: reference to 'display' is ambiguous
```

```
display();
```

The above issue can be resolved by using the class resolution operator with the function. In the above example, the derived class code can be rewritten as:

```
class C : public A, public B
{
    void view()
    {
        A::display();    // Calling the display() function of class A.
        B::display();    // Calling the display() function of class B.
    }
};
```

An ambiguity can also occur in single inheritance.

Consider the following situation:

```
class A
{
    public:
```



```

void display()

{

    cout<<"Class A";

}

};

class B

{

    public:

    void display()

    {

        cout<<"Class B";

    }

};

```

In the above case, the function of the derived class overrides the method of the base class. Therefore, call to the display() function will simply call the function defined in the derived class. If we want to invoke the base class function, we can use the class resolution operator.

```

int main()

{

```

```

    B b;

    b.display();           // Calling the display() function of B class.

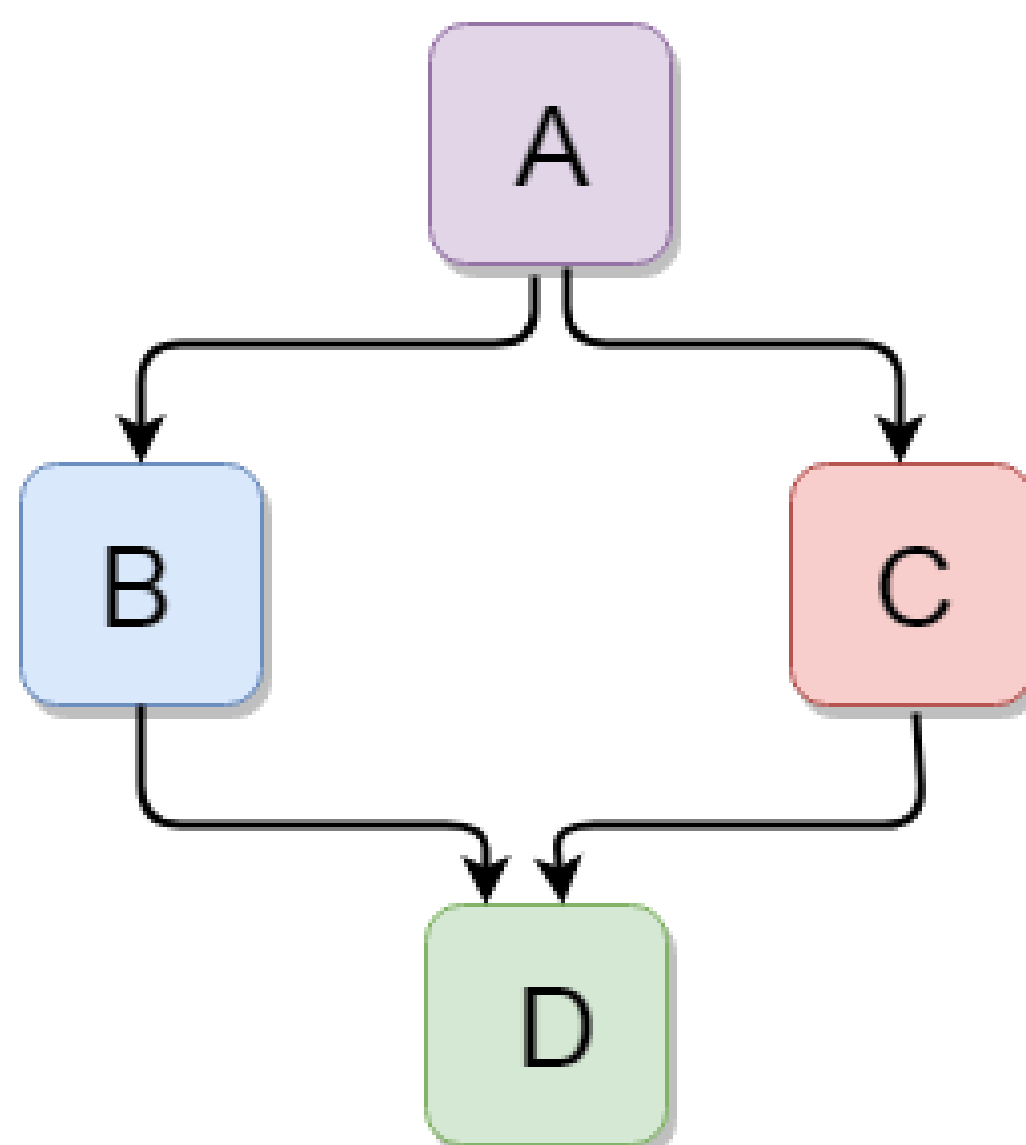
    b.B :: display();      // Calling the display() function defined in B class.

}

```

C++ Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.



Let's see a simple example:

```

#include <iostream>

using namespace std;

class A
{

```



```

protected:

int a;

public:

void get_a( )

{

    std::cout << "Enter the value of 'a' : " << std::endl;

    cin>>a;

}

};

class B : public A

{

protected:

int b;

public:

void get_b( )

{

    std::cout << "Enter the value of 'b' : " << std::endl;

    cin>>b;

```

```

    }

};

class C

{

    protected:

        int c;

    public:

        void get_c()

        {

            std::cout << "Enter the value of c is : " << std::endl;

            cin>>c;

        }

};

class D : public B, public C

{

    protected:

        int d;

    public:

```

```

void mul( )

{

    get_a();

    get_b();

    get_c();

    std::cout << "Multiplication of a,b,c is : " <<a*b*c<< std::endl;

}

};

int main( )

{

    D d;

    d.mul();

    return 0;

}

```

Output:

Enter the value of 'a' :

10

Enter the value of 'b' :

20

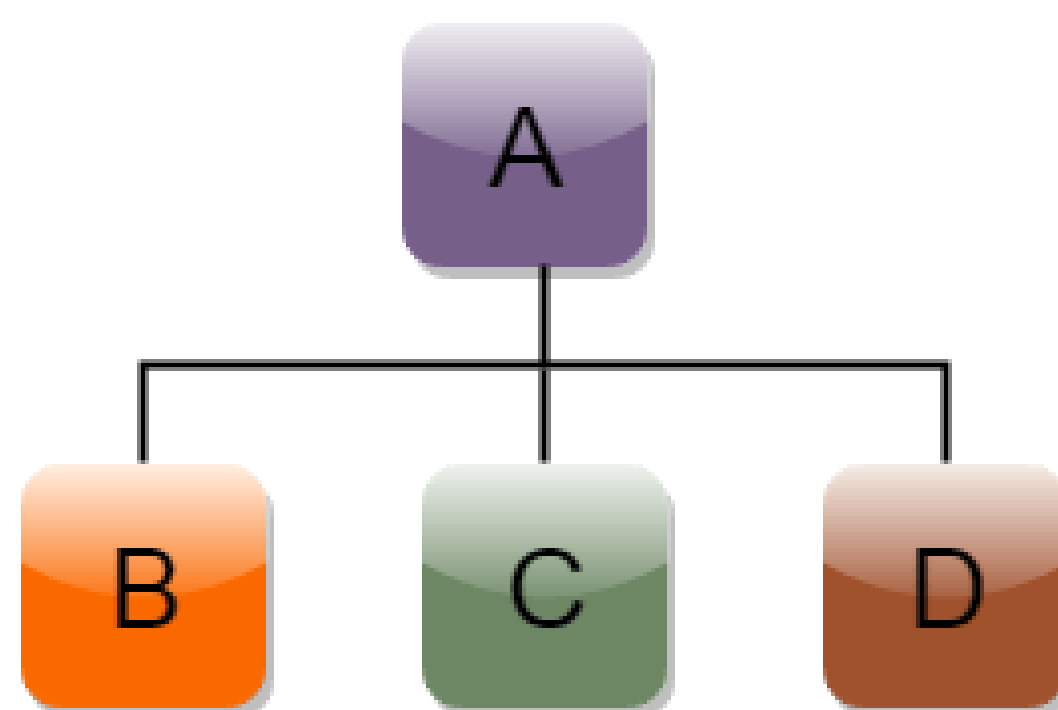
Enter the value of c is :

30

Multiplication of a,b,c is : 6000

C++ Hierarchical Inheritance

Hierarchical inheritance is defined as the process of deriving more than one class from a baseclass.



Syntax of Hierarchical inheritance:

```
class A
{
    // body of the class A.
}

class B : public A
{
    // body of class B.
```

```
}
```

```
class C : public A
```

```
{
```

```
    // body of class C.
```

```
}
```

```
class D : public A
```

```
{
```

```
    // body of class D.
```

```
}
```

Let's see a simple example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Shape          // Declaration of base class.
```

```
{
```

```
    public:
```

```
    int a;
```

```
    int b;
```

```
    void get_data(int n,int m)
```

```

{

    a= n;

    b = m;

}

};

class Rectangle : public Shape // inheriting Shape class

{

    public:

    int rect_area( )

    {

        int result = a*b;

        return result;

    }

};

class Triangle : public Shape // inheriting Shape class

{

    public:

    int triangle_area( )

```



```

{

    float result = 0.5 * a * b;

    return result;

}

};

int main()

{

    Rectangle r;

    Triangle t;

    int length, breadth, base, height;

    std::cout << "Enter the length and breadth of a rectangle: " << std::endl;

    cin >> length >> breadth;

    r.get_data(length, breadth);

    int m = r.rect_area();

    std::cout << "Area of the rectangle is : " << m << std::endl;

    std::cout << "Enter the base and height of the triangle: " << std::endl;

    cin >> base >> height;

    t.get_data(base, height);

```

```

float n = t.triangle_area();

std::cout <<"Area of the triangle is : " << n<<std::endl;

return 0;

}

```

Output:

Enter the length and breadth of a rectangle:

23

20

Area of the rectangle is : 460

Enter the base and height of the triangle:

2

5

Area of the triangle is : 5

C++ Aggregation (HAS-ARelationship)

In C++, aggregation is a process in which one class defines another class as any entity reference. It is another way to reuse the class. It is a form of association that represents HAS-ARelationship.

C++ Aggregation Example

Let's see an example of aggregation where Employee class has the reference of Address class as data member. In such way, it can reuse the members of Address class.


```
#include <iostream>
```

```
using namespace std;
```

```
class Address {
```

```
    public:
```

```
    string addressLine, city, state;
```

```
    Address(string addressLine, string city, string state)
```

```
{
```

```
    this->addressLine = addressLine;
```

```
    this->city = city;
```

```
    this->state = state;
```

```
}
```

```
};
```

```
class Employee
```

```
{
```

```
    private:
```

```
    Address* address; //Employee HAS-A Address
```

```
    public:
```

```
    int id;
```

```

    string name;

    Employee(int id, string name, Address* address)

    {

        this->id = id;

        this->name = name;

        this->address = address;

    }

    void display()

    {

        cout<<id <<" "<<name<<" "<<

            address->addressLine<<" "<< address->city<<" "<<address->state

        <<endl;

    }

};

int main(void) {

    Address a1= Address("C-146, Sec-15","Noida","UP");

    Employee e1 = Employee(101,"Nakul",&a1);

    e1.display();

```

```
return 0;  
  
}
```

Output:

101 Nakul C-146, Sec-15 Noida UP

C++ Polymorphism

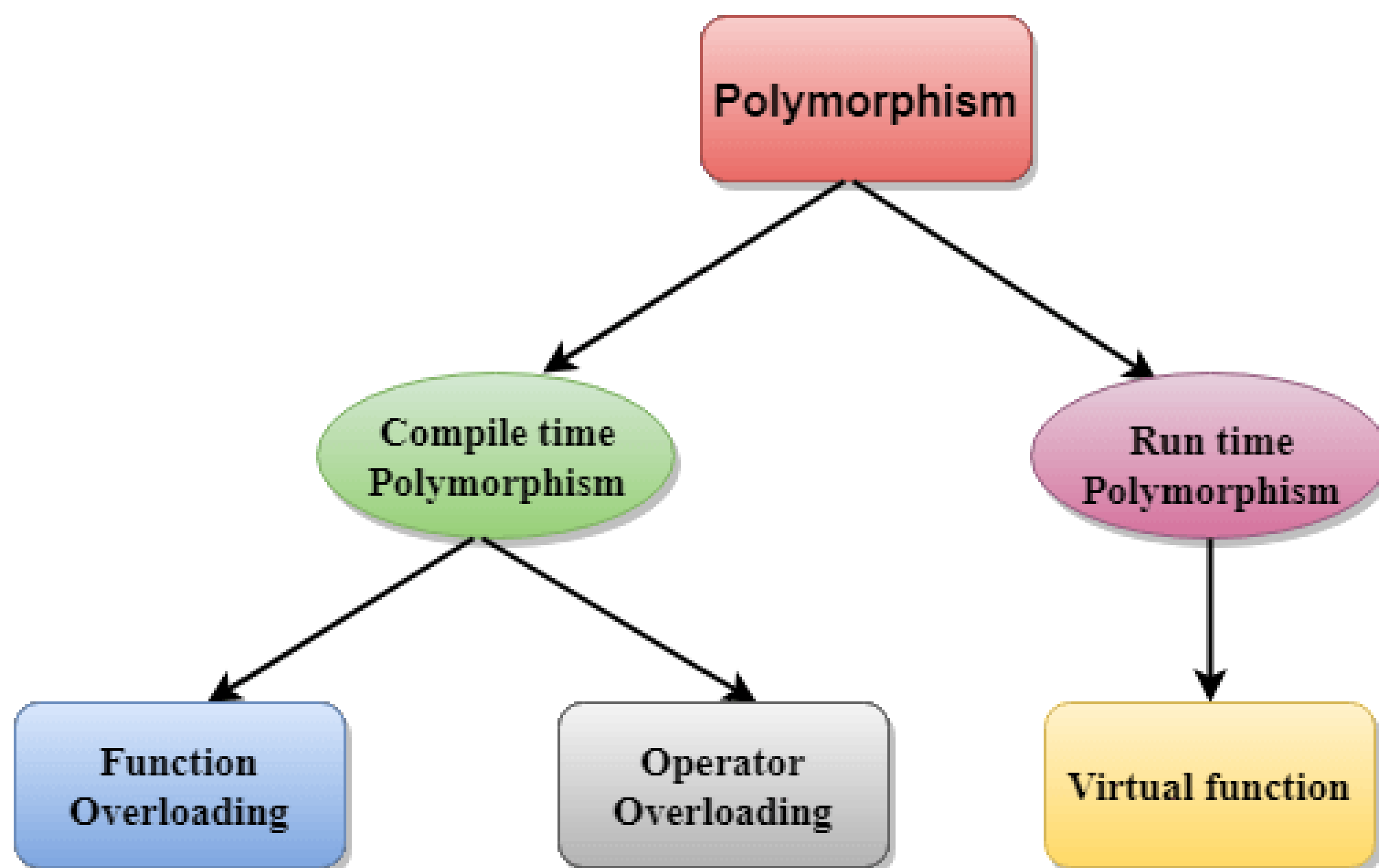
C++ Polymorphism

The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a Greek word. In object-oriented programming, we use 3 main concepts: inheritance, encapsulation, and polymorphism.

Real Life Example Of Polymorphism

Let's consider a real-life example of polymorphism. A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market. Here, a single person is behaving differently according to the situations.

There are two types of polymorphism in C++:



Compile time polymorphism: The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Now, let's consider the case where function name and prototype is same.

```
class A // base class declaration.

{

    int a;

    public:

    void display()

    {

        cout<< "Class A ";
```

```

    }

};

class B : public A           // derived class declaration.

{

    int b;

    public:

    void display()

    {

        cout<<"Class B";

    }

};

```

In the above case, the prototype of display() function is the same in both the **base and derived class**. Therefore, the static binding cannot be applied. It would be great if the appropriate function is selected at the run time. This is known as **run time polymorphism**.

Run time polymorphism: Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

C++ Runtime Polymorphism Example

Let's see a simple example of run time polymorphism in C++.

// an example without the virtual keyword.

```
#include <iostream>

using namespace std;

class Animal {

    public:

    void eat(){

        cout<<"Eating...";

    }

};

class Dog: public Animal

{

    public:

    void eat()

    {

        cout<<"Eating bread...";

    }

};

int main(void) {

    Dog d = Dog();
```

```

    d.eat();

    return 0;

}

```

Output:

```
Eating bread...
```

C++ Runtime Polymorphism Example: By using two derived class

Let's see another example of run time polymorphism in C++ where we are having two derived classes.

// an example with virtual keyword.

```

#include <iostream>

using namespace std;

class Shape {                                // base class

public:

    virtual void draw() {                    // virtual function

        cout<<"drawing..."<<endl;

    }

};

class Rectangle: public Shape                // inheriting Shape class.

```



```

{

public:

void draw()

{

    cout<<"drawing rectangle..."<<endl;

}

};

class Circle: public Shape           // inheriting Shape class.

{

public:

void draw()

{

    cout<<"drawing circle..."<<endl;

}

};

int main(void) {

    Shape *s;                       // base class pointer.

    Shape sh;                       // base class object.

```

```

    Rectangle rec;

    Circle cir;

    s=&sh;

    s->draw();

    s=&rec;

    s->draw();

    s=?

    s->draw();

}

```

Output:

```

drawing...
drawing rectangle...
drawing circle...

```

Runtime Polymorphism with Data Members

Runtime Polymorphism can be achieved by data members in C++. Let's see an example where we are accessing the field by reference variable which refers to the instance of derived class.

```

#include <iostream>

using namespace std;

```

```

class Animal {                                // base class declaration.

    public:

    string color = "Black";

};

class Dog: public Animal                      // inheriting Animal class.

{

    public:

    string color = "Grey";

};

int main(void) {

    Animal d= Dog( );

    cout<<d.color;

}

```

Output:

Black

C++ Overloading (Function and Operator)

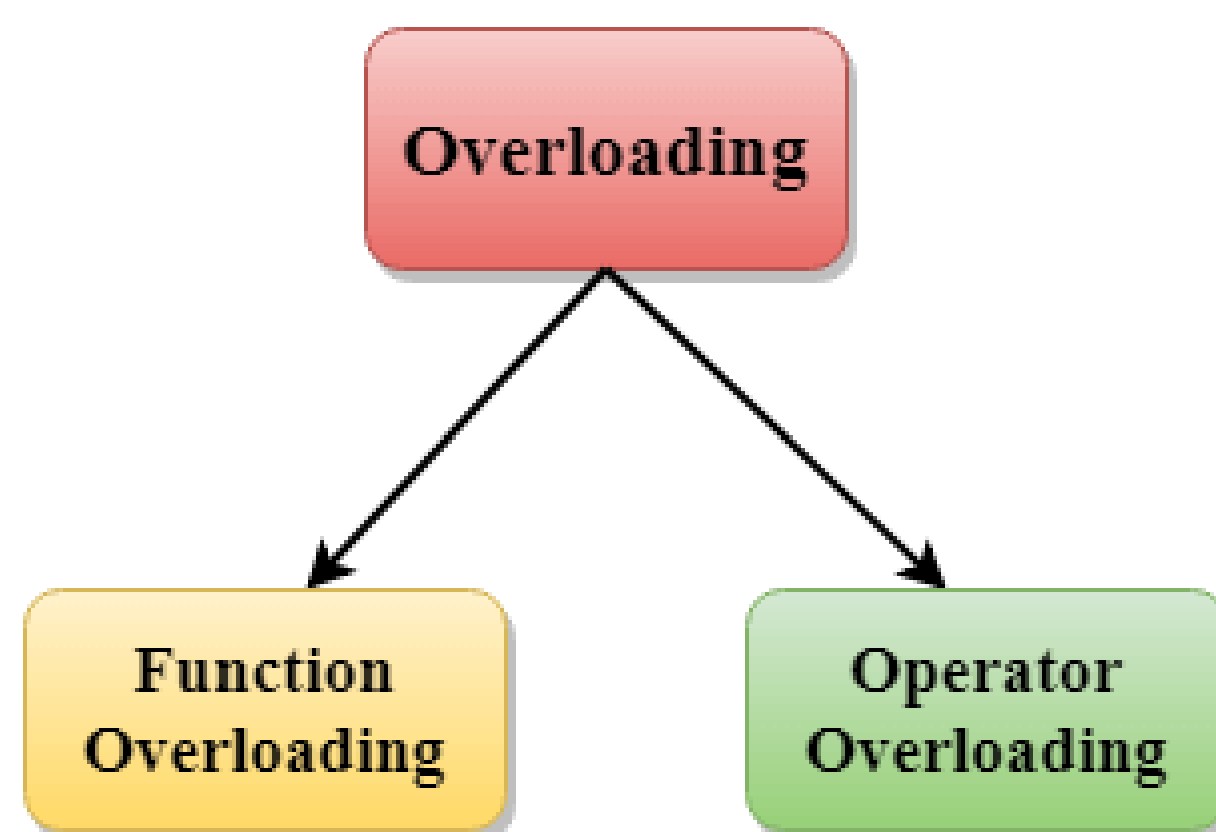
If we create two or more members having the same name but different in number or type of parameter, it is known as C++ overloading. In C++, we can overload:

- methods,
- constructors, and
- indexed properties

It is because these members have parameters only.

Types of overloading in C++ are:

- Function overloading
- Operator overloading



C++ Function Overloading

Function Overloading is defined as the process of having two or more functions with the same name, but different parameters, known as function overloading in C++. In function overloading, the function is redefined by using either different types of arguments or a different number of arguments. It is only through these differences that the compiler can differentiate between the functions.

The **advantage** of Function overloading is that it increases the readability of the program because you don't need to use different names for the same action.

C++ Function Overloading Example

Let's see the simple example of function overloading where we are changing number of arguments of add() method.

// program of function overloading when number of arguments vary.

```
#include <iostream>

using namespace std;

class Cal {

    public:

    static int add(int a,int b){

        return a + b;

    }

    static int add(int a, int b, int c)

    {

        return a + b + c;

    }

};
```

```

int main(void) {

    Cal C;                // class object declaration.

    cout<<C.add(10, 20)<<endl;

    cout<<C.add(12, 20, 23);

    return 0;

}

```

Output:

30

55

Let's see the simple example when the type of the arguments vary.

// Program of function overloading with different types of arguments.

```
#include<iostream>
```

```
using namespace std;
```

```
int mul(int,int);
```

```
float mul(float,int);
```

```
int mul(int a,int b)
```



```

{

    return a*b;

}

float mul(double x, int y)

{

    return x*y;

}

int main()

{

    int r1 = mul(6,7);

    float r2 = mul(0.2,3);

    std::cout << "r1 is : " <<r1<< std::endl;

    std::cout <<"r2 is : " <<r2<< std::endl;

    return 0;

}

```

Output:

```

r1 is : 42

r2 is : 0.6

```

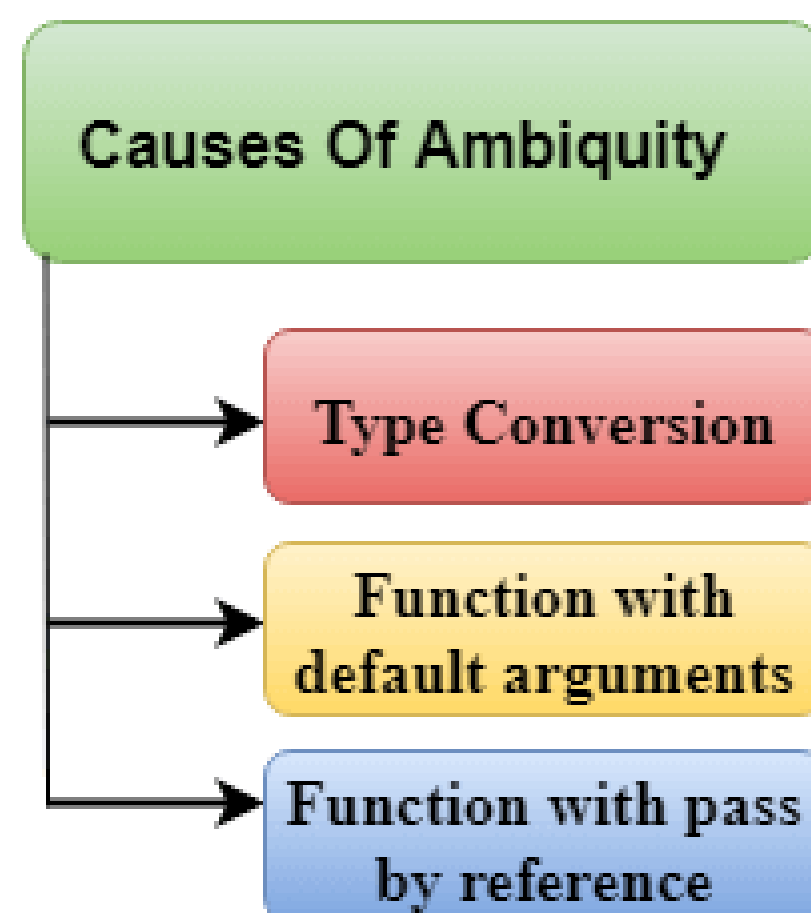
Function Overloading and Ambiguity

When the compiler is unable to decide which function is to be invoked among the overloaded function, this situation is known as **function overloading**.

When the compiler shows the ambiguity error, the compiler does not run the program.

Causes of Function Overloading:

- Type Conversion.
- Function with default arguments.
- Function with pass by reference.



Type Conversion:

Let's see a simple example.

```
#include <iostream>
```

```
using namespace std;
```



```

void fun(int);

void fun(float);

void fun(int i)
{
    std::cout << "Value of i is : " << i << std::endl;
}

void fun(float j)
{
    std::cout << "Value of j is : " << j << std::endl;
}

int main()
{
    fun(12);

    fun(1.2);

    return 0;
}

```

The above example shows an error "call of overloaded 'fun(double)' is ambiguous". The fun(10) will call the first function. The fun(1.2) calls the second function according to our

prediction. But, this does not refer to any function as in C++, all the floating point constants are treated as double not as a float. If we replace float to double, the program works. Therefore, this is a type conversion from float to double.

Function with Default Arguments

Let's see a simple example.

```
#include<iostream>

using namespace std;

void fun(int);

void fun(int,int);

void fun(int i)

{

    std::cout << "Value of i is : " <<i<< std::endl;

}

void fun(int a,int b=9)

{

    std::cout << "Value of a is : " <<a<< std::endl;

    std::cout << "Value of b is : " <<b<< std::endl;

}
```

```

int main()

{

    fun(12);


    return 0;

}

```

The above example shows an error "call of overloaded 'fun(int)' is ambiguous". The fun(int a, int b=9) can be called in two ways: first is by calling the function with one argument, i.e., fun(12) and another way is calling the function with two arguments, i.e., fun(4,5). The fun(int i) function is invoked with one argument. Therefore, the compiler could not be able to select among fun(int i) and fun(int a, int b=9).

Function with pass by reference

Let's see a simple example.

```

#include <iostream>

using namespace std;

void fun(int);

void fun(int &);

int main()

{

```



```

int a=10;

fun(a); // error, which f()?

return 0;

}

void fun(int x)

{

std::cout << "Value of x is : " <<x<< std::endl;

}

void fun(int &b)

{

std::cout << "Value of b is : " <<b<< std::endl;

}

```

The above example shows an error "call of overloaded 'fun(int&)' is ambiguous". The first function takes one integer argument and the second function takes a reference parameter as an argument. In this case, the compiler does not know which function is needed by the user as there is no syntactical difference between the fun(int) and fun(int &).

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used

to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector (.)
- member pointer selector (*)
- ternary operator (?:)

Syntax of Operator Overloading

```
return_type class_name :: operator op(argument_list)

{

    // body of the function.

}
```

Where the **return type** is the type of value returned by the function.

class_name is the name of the class.

`operator op` is an operator function where `op` is the operator being overloaded, and the `operator` is the keyword.

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains at least one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

C++ Operators Overloading Example

Let's see the simple example of operator overloading in C++. In this example, void operator ++ () operator function is defined (inside Test class).

```
// program to overload the unary operator ++.
```

```
#include <iostream>
```

```
using namespace std;
```



```

class Test
{

private:

    int num;

public:

    Test(): num(8){}

    void operator ++()    {

        num = num+2;

    }

    void Print() {

        cout<<"The Count is: "<<num;

    }

};

int main()

{

    Test tt;

    ++tt; // calling of a function "void operator ++()"

    tt.Print();

```

```
    return 0;

}
```

Output:

The Count is: 10

Let's see a simple example of overloading the binary operators.

// program to overload the binary operators.

```
#include <iostream>
```

```
using namespace std;
```

```
class A
```

```
{
```

```
    int x;
```

```
    public:
```

```
        A(){}
```

```
        A(int i)
```

```
{
```

```
    x=i;
```

```
}
```

```
    void operator+(A);
```



```

void display();

};

void A::operator+(A a)

{

    int m = x+a.x;

    cout<<"The result of the addition of two objects is : "<<m;

}

int main()

{

    A a1(5);

    A a2(4);

    a1+a2;

    return 0;

}

```

Output:

The result of the addition of two objects is : 9

C++ Function Overriding

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

C++ Function Overriding Example

Let's see a simple example of Function overriding in C++. In this example, we are overriding the eat() function.

```
#include <iostream>

using namespace std;

class Animal {

    public:

    void eat(){

        cout<<"Eating...";

    }

};

class Dog: public Animal

{

    public:
```

```

void eat()

{

    cout<<"Eating bread...";

}

};

int main(void) {

    Dog d = Dog();

    d.eat();

    return 0;

}

```

Output:

Eating bread...

C++ virtual function

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.

- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Late binding or Dynamic linkage

In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.

- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
```

```
using namespace std;
```

```
class A
```

```
{
```

```
    int x=5;
```

```
    public:
```

```
    void display()
```

```
{
```

```
    std::cout << "Value of x is : " << x<<std::endl;
```

```
}
```

```
};
```

```
class B: public A
```

```
{
```

```

int y = 10;

public:

void display()

{

    std::cout << "Value of y is : " <<y<< std::endl;

}

};

int main()

{

    A *a;

    B b;

    a = &b;

    a->display();

    return 0;

}

```

Output:

Value of x is : 5

In the above example, * a is the base class pointer. The pointer can only access the base class members but not the members of the derived class. Although C++ permits the base pointer to point to any object derived from the base class, it cannot directly access the members of the derived class. Therefore, there is a need for virtual function which allows the base pointer to access the members of the derived class.

C++ virtual function Example

Let's see the simple example of C++ virtual function used to invoke the derived class in a program.

```
#include <iostream>

{

public:

virtual void display( )

{

    cout << "Base class is invoked"<<endl;

}

};

class B:public A

{

public:
```

```

void display( )

{

    cout << "Derived Class is invoked"<<endl;

}

};

int main( )

{

    A* a;  //pointer of base class

    B b;   //object of derived class

    a = &b;

    a->display( ); //Late Binding occurs

}

```

Output:

Derived Class is invoked

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "~~do-nothing~~" function.

- The "do-nothing" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure virtual function can be defined as:

```
virtual void display() = 0;
```

Let's see a simple example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Base
```

```
{
```

```
    public:
```

```
    virtual void show() = 0;
```

```
};
```

```
class Derived : public Base
```

```
{
```

```

public:

void show( )

{

    std::cout << "Derived class is derived from the base class." << std::endl;

}

};

int main( )

{

    Base *bptr;

    //Base b;

    Derived d;

    bptr = &d;

    bptr->show( );

    return 0;

}

```

Output:

Derived class is derived from the base class.

In the above example, the base class contains the pure virtual function. Therefore, the base class is an abstract base class. We cannot create the object of the base class.

C++ Abstraction

Interfaces in C++ (Abstract Classes)

Abstract classes are the way to achieve abstraction in C++. Abstraction in C++ is the process to hide the internal details and showing functionality only. Abstraction can be achieved by two ways:

1. Abstract class
2. Interface

Abstract class and interface both can have abstract methods which are necessary for abstraction.

C++ Abstract class

In C++ class is made abstract by declaring at least one of its functions as `<>strong>pure` virtual function. A pure virtual function is specified by placing `"= 0"` in its declaration. Its implementation must be provided by derived classes.

Let's see an example of abstract class in C++ which has one abstract method `draw()`. Its implementation is provided by derived classes: Rectangle and Circle. Both classes have different implementation.

```
#include <iostream>
```



```
using namespace std;

class Shape

{

    public:

    virtual void draw()=0;

};

class Rectangle : Shape

{

    public:

    void draw()

    {

        cout << "drawing rectangle..." << endl;

    }

};

class Circle : Shape

{

    public:

    void draw()
```

```

    {

        cout <<"drawing circle..." <<endl;

    }

};

int main( ) {

    Rectangle rec;

    Circle cir;

    rec.draw( );

    cir.draw( );

    return 0;

}

```

Output:

drawing rectangle...

drawing circle...

Data Abstraction in C++

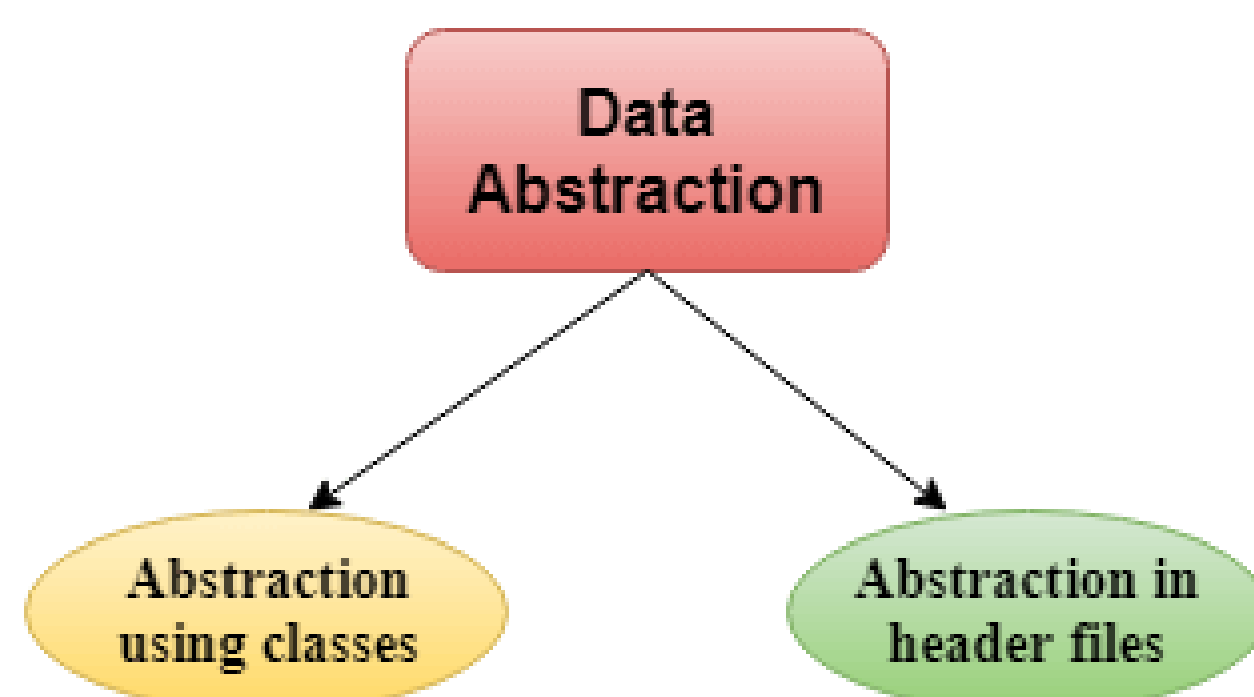
- Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program.

- Data Abstraction is a programming technique that depends on the separation of the interface and implementation details of the program.
- Let's take a real life example of AC, which can be turned ON or OFF, change the temperature, change the mode, and other external components such as fan, swing. But, we don't know the internal details of the AC, i.e., how it works internally. Thus, we can say that AC separates the implementation details from the external interface.
- C++ provides a great level of abstraction. For example, `pow()` function is used to calculate the power of a number without knowing the algorithm the function follows.

In C++ program if we implement class with private and public members then it is an example of data abstraction.

Data Abstraction can be achieved in two ways:

- Abstraction using classes
- Abstraction in header files.



Abstraction using classes: An abstraction can be achieved using classes. A class is used to group all the data members and member functions into a single unit by using the access

specifiers. A class has the responsibility to determine which data member is to be visible outside and which is not.

Abstraction in header files: Another type of abstraction is header file. For example, `pow()` function available is used to calculate the power of a number without actually knowing which algorithm function uses to calculate the power. Thus, we can say that header files hides all the implementation details from the user.

Access Specifiers Implement Abstraction:

- **Public specifier:** When the members are declared as public, members can be accessed anywhere from the program.
- **Private specifier:** When the members are declared as private, members can only be accessed only by the member functions of the class.

Let's see a simple example of abstraction in header files.

```
// program to calculate the power of a number.
```

```
#include <iostream>
```

```
#include <math.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int n = 4;
```

```

int power = 3;

int result = pow(n,power);    // pow(n,power) is the power function

std::cout << "Cube of n is : " << result << std::endl;

return 0;

}

```

Output:

Cube of n is : 64

In the above example, pow() function is used to calculate 4 raised to the power 3. The pow() function is present in the math.h header file in which all the implementation details of the pow() function is hidden.

Let's see a simple example of data abstraction using classes.

```

#include <iostream>

using namespace std;

class Sum
{

private: int x, y, z; // private variables

public:

void add()

```



```

{

cout<<"Enter two numbers: ";

cin>>x>>y;

z= x+y;

cout<<"Sum of two number is: "<<z<<endl;

}

};

int main()

{

Sum sm;

sm.add();

return 0;

}

```

Output:

```

Enter two numbers:

3

6

Sum of two number is: 9

```

In the above example, abstraction is achieved using classes. A class 'Sum' contains the private members x, y and z are only accessible by the member functions of the class.

Advantages Of Abstraction:

- Implementation details of the class are protected from the inadvertent user level errors.
- A programmer does not need to write the low level code.
- Data Abstraction avoids the code duplication, i.e., programmer does not have to undergo the same tasks every time to perform the similar operation.
- The main aim of the data abstraction is to reuse the code and the proper partitioning of the code across the classes.
- Internal implementation can be changed without affecting the user level code.

C++ Namespaces

C++ Namespaces

Namespaces in C++ are used to organize too many classes so that it can be easy to handle the application.

For accessing the class of a namespace, we need to use namespace::classname.

We can use **using** keyword so that we don't have to use complete name all the time.

In C++, global namespace is the root namespace. The global::std will always refer to the namespace "std" of C++ Framework.

C++ namespace Example

Let's see the simple example of namespace which include variable and functions.

```
#include <iostream>

using namespace std;

namespace First {

    void sayHello() {

        cout<<"Hello First Namespace"<<endl;

    }

}

namespace Second {

    void sayHello() {

        cout<<"Hello Second Namespace"<<endl;

    }

}

int main()

{

    First::sayHello();

    Second::sayHello();

}
```



```
return 0;
```

```
}
```

Output:

```
Hello First Namespace
```

```
Hello Second Namespace
```

C++ namespace example: by using keyword

Let's see another example of namespace where we are using "using" keyword so that we don't have to use complete name for accessing a namespace program.

```
#include <iostream>
```

```
using namespace std;
```

```
namespace First{
```

```
void sayHello( ){
```

```
    cout << "Hello First Namespace" << endl;
```

```
}
```

```
}
```

```
namespace Second{
```

```
void sayHello( ){
```

```
    cout << "Hello Second Namespace" << endl;
```

```
    }  
}  
  
using namespace First;  
  
int main () {  
  
    sayHello();  
  
    return 0;  
}
```

Output:

Hello First Namespace

C++ Templates

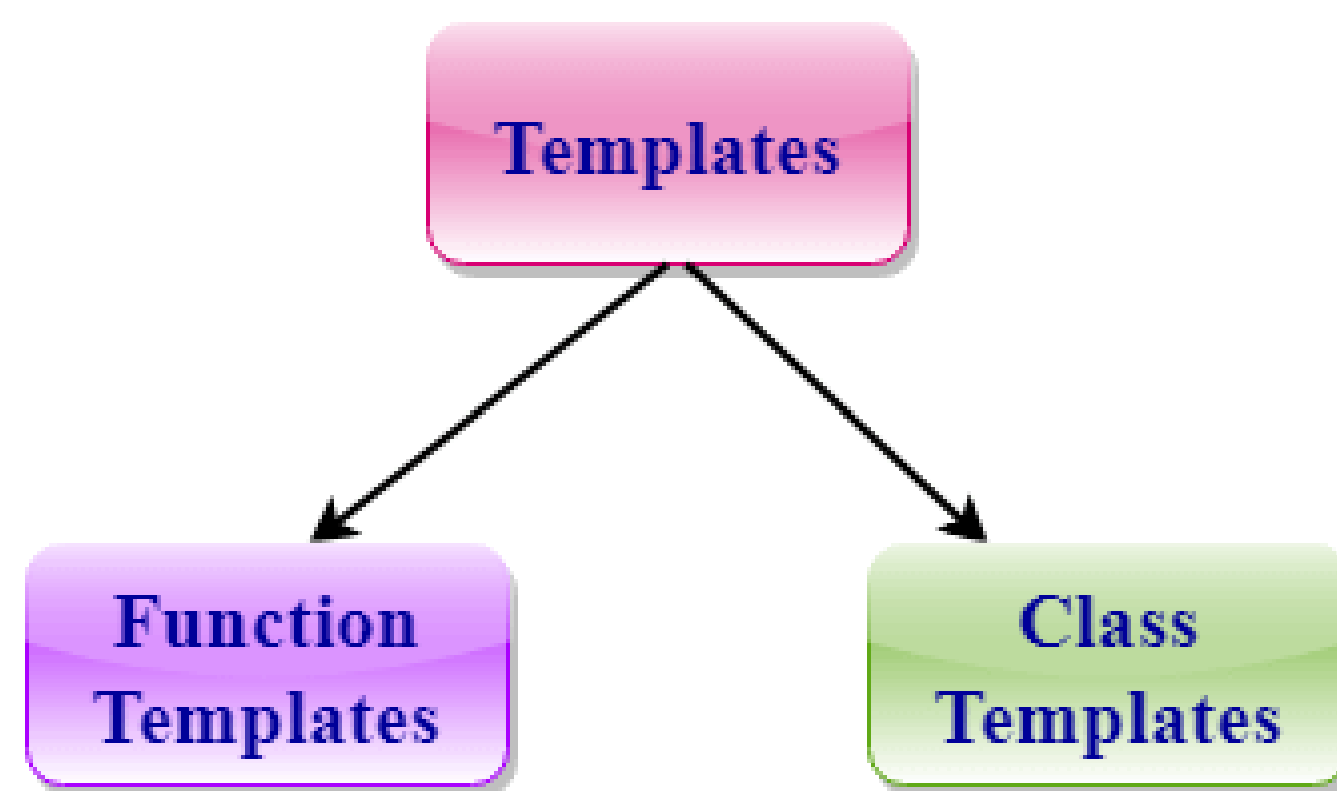
C++ Templates

A C++ template is a powerful feature added to C++. It allows you to define the generic classes and generic functions and thus provides support for generic programming. Generic programming is a technique where generic types are used as parameters in algorithms so that they can work for a variety of data types.

Templates can be represented in two ways:

- Function templates

- Class templates



Function Templates:

We can define a template for a function. For example, if we have an `add()` function, we can create versions of the `add` function for adding the `int`, `float` or `double` type values.

Class Template:

We can define a template for a class. For example, a class template can be created for the array class that can accept the array of various types such as `int` array, `float` array or `double` array.

Function Template

- Generic functions use the concept of a function template. Generic functions define a set of operations that can be applied to the various types of data.
- The type of the data that the function will operate on depends on the type of the data passed as a parameter.

- For example, Quick sorting algorithm is implemented using a generic function, it can be implemented to an array of integers or array of floats.
- A Generic function is created by using the keyword template. The template defines what function will do.

Syntax of Function Template

```
template < class T type> ret_type func_name(parameter_list)

{

    // body of function.

}
```

Where T type: It is a placeholder name for a data type used by the function. It is used within the function definition. It is only a placeholder that the compiler will automatically replace this placeholder with the actual data type.

class: A class keyword is used to specify a generic type in a template declaration.

Let's see a simple example of a function template:

```
#include <iostream>

using namespace std;

template<class T> T add(T &a,T &b)

{

    T result = a+b;
```

```

    return result;

}

int main()

{

    int i =2;

    int j =3;

    float m = 2.3;

    float n = 1.2;

    cout<<"Addition of i and j is :"<<add(i,j);

    cout<<"\n";

    cout<<"Addition of m and n is :"<<add(m,n);

    return 0;

}

```

Output:

Addition of i and j is :5

Addition of m and n is :3.5

In the above example, we create the function template which can perform the addition operation on any type either it can be integer, float or double.

Function Templates with Multiple Parameters

We can use more than one generic type in the template function by using the comma to separate the list.

Syntax

```
template<class T1, class T2,.....>
```

```
return_type function_name (arguments of type T1, T2....)
```

```
{
```

```
    // body of function.
```

```
}
```

In the above syntax, we have seen that the template function can accept any number of arguments of a different type.

Let's see a simple example:

```
#include <iostream>
```

```
using namespace std;
```

```
template<class X, class Y> void fun(X a, Y b)
```

```
{
```

```
    std::cout << "Value of a is : " <<a<< std::endl;
```

```
    std::cout << "Value of b is : " <<b<< std::endl;
```

```

}

int main()

{

    fun(15,12.3);

    return 0;

}

```

Output:

Value of a is : 15

Value of b is : 12.3

In the above example, we use two generic types in the template function, i.e., X and Y.

Overloading a Function Template

We can overload the generic function means that the overloaded template functions can differ in the parameter list.

Let's understand this through a simple example:

```

#include <iostream>

using namespace std;

template<class X> void fun(X a)

{

```

```

        std::cout << "Value of a is : " <<a<< std::endl;

    }

    template<class X,class Y> void fun(X b ,Y c)

    {

        std::cout << "Value of b is : " <<b<< std::endl;

        std::cout << "Value of c is : " <<c<< std::endl;

    }

    int main()

    {

        fun(10);

        fun(20,30.5);

        return 0;

    }

```

Output:

Value of a is : 10

Value of b is : 20

Value of c is : 30.5

In the above example, template of fun() function is overloaded.

Restrictions of Generic Functions

Generic functions perform the same operation for all the versions of a function except the data type differs. Let's see a simple example of an overloaded function which cannot be replaced by the generic function as both the functions have different functionalities.

Let's understand this through a simple example:

```
#include <iostream>

using namespace std;

void fun(double a)

{

    cout<<"value of a is : "<<a<<"\n";

}

void fun(int b)

{

    if(b%2==0)

    {

        cout<<"Number is even";

    }

    else
```

```

    {

        cout<<"Number is odd";

    }

}

int main()

{

    fun(4.6);

    fun(6);

    return 0;

}

```

Output:

```
value of a is : 4.6
```

```
Number is even
```

In the above example, we overload the ordinary functions. We cannot overload the generic functions as both the functions have different functionalities. First one is displaying the value and the second one determines whether the number is even or not.

CLASSTEMPLATE

Class Template can also be defined similarly to the Function Template. When a class uses the concept of Template, then the class is known as generic class.

Syntax

```
template<class T type>
```

```
class class_name
```

```
{
```

```
·
```

```
·
```

```
}
```

T type is a placeholder name which will be determined when the class is instantiated. We can define more than one generic data type using a comma-separated list. The T type can be used inside the class body.

Now, we create an instance of a class

```
class_name<type> ob;
```

where class_name: It is the name of the class.

type: It is the type of the data that the class is operating on.

ob: It is the name of the object.

Let's see a simple example:


```
#include <iostream>
```

```
using namespace std;
```

```
template<class T>
```

```
class A
```

```
{
```

```
    public:
```

```
        T num1 = 5;
```

```
        T num2 = 6;
```

```
        void add()
```

```
{
```

```
    std::cout << "Addition of num1 and num2 : " << num1+num2<<std::endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    A<int> d;
```

```
d.add();  
  
return 0;  
  
}
```

Output:

Addition of num1 and num2 : 11

In the above example, we create a template for class A. Inside the main() method, we create the instance of class A named as, 'd'.

CLASS TEMPLATE WITH MULTIPLE PARAMETERS

We can use more than one generic data type in a class template, and each generic data type is separated by the comma.

Syntax

```
template<class T1, class T2, .....>  
  
class class_name  
  
{  
  
    // Body of the class.  
  
}
```

Let's see a simple example when class template contains two generic data types.

```
#include <iostream>
```



```
using namespace std;

template<class T1, class T2>

class A

{

    T1 a;

    T2 b;

    public:

    A(T1 x,T2 y)

    {

        a = x;

        b = y;

    }

    void display()

    {

        std::cout << "Values of a and b are : " << a<<" ,"<<b<<std::endl;

    }

};

int main()
```

```

{

    A<int,float> d(5,6.5);

    d.display();

    return 0;

}

```

Output:

Values of a and b are : 5,6.5

Nontype Template Arguments

The template can contain multiple arguments, and we can also use the non-type arguments. In addition to the type T argument, we can also use other types of arguments such as strings, function names, constant expression and built-in types.

Let's see the following example:

```

template<class T, int size>

class array

{

    T arr[size];    // automatic array initialization.

};

```

In the above case, the nontype template argument is size and therefore, template supplies the size of the array as an argument.

Arguments are specified when the objects of a class are created:

```
array<int, 15> t1;           // array of 15 integers.
```

```
array<float, 10> t2;        // array of 10 floats.
```

```
array<char, 4> t3;          // array of 4 chars.
```

Let's see a simple example of nontype template arguments.

```
#include <iostream>

using namespace std;

template<class T, int size>

class A

{

public:

    T arr[size];

    void insert()

    {

        int i = 1;

        for (int j = 0; j < size; j++)

        {

            arr[j] = i;
```



```

        i++;

    }

}

void display()

{

    for(int i=0;i<size;i++)

    {

        std::cout << arr[i] << " ";

    }

}

};

int main()

{

    A<int,10> t1;

    t1.insert();

    t1.display();

    return 0;

}

```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

In the above example, the class template is created which contains the nontype template argument, i.e., size. It is specified when the object of class 'A' is created.

Points to Remember

- C++ supports a powerful feature known as a template to implement the concept of generic programming.
- A template allows us to create a family of classes or family of functions to handle different data types.
- Template classes and functions eliminate the code duplication of different data types and thus makes the development easier and faster.
- Multiple parameters can be used in both class and function template.
- Template functions can also be overloaded.
- We can also use nontype arguments such as built-in or derived data types as template arguments.

C++ Strings

C++ Strings

In C++, string is an object of `std::string` class that represents sequence of characters. We can perform many operations on strings such as concatenation, comparison, conversion etc.

C++ String Example

Let's see the simple example of C++ string.

```
#include <iostream>

using namespace std;

int main( ) {

    string s1 = "Hello";

    char ch[] = { 'C', '+', '+' };

    string s2 = string(ch);

    cout<<s1<<endl;

    cout<<s2<<endl;

}
```

Output:

Hello

C++

C++ String Compare Example

Let's see the simple example of string comparison using strcmp() function.

```
#include <iostream>

#include <cstring>

using namespace std;

int main ( )

{

    char key[ ] = "mango";

    char buffer[50];

    do {

        cout<<"What is my favourite fruit? ";

        cin>>buffer;

    } while ( strcmp (key,buffer) != 0);

    cout<<"Answer is correct!!"<<endl;

    return 0;

}
```

Output:

What is my favourite fruit? apple

What is my favourite fruit? banana

What is my favourite fruit? mango

Answer is correct!!

C++ String Concat Example

Let's see the simple example of string concatenation using `strcat()` function.

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char key[25], buffer[25];
```

```
    cout << "Enter the key string: ";
```

```
    cin.getline(key, 25);
```

```
    cout << "Enter the buffer string: ";
```

```
    cin.getline(buffer, 25);
```

```
    strcat(key, buffer);
```



```

cout << "Key = " << key << endl;

cout << "Buffer = " << buffer<<endl;

return 0;

}

```

Output:

```

Enter the key string: Welcome to
Enter the buffer string:  C++ Programming.
Key = Welcome to C++ Programming.
Buffer =  C++ Programming.

```

C++ String Copy Example

Let's see the simple example of copy the string using strcpy() function.

```

#include <iostream>

#include <cstring>

using namespace std;

int main()

{

    char key[25], buffer[25];

    cout << "Enter the key string: ";

```

```

cin.getline(key, 25);

strcpy(buffer, key);

cout << "Key = " << key << endl;

cout << "Buffer = " << buffer << endl;

return 0;

}

```

Output:

Enter the key string: C++ Tutorial

Key = C++ Tutorial

Buffer = C++ Tutorial

C++ String Length Example

Let's see the simple example of finding the string length using `strlen()` function.

```

#include <iostream>

#include <cstring>

using namespace std;

int main()

{

    char ary[] = "Welcome to C++ Programming";

```

```
cout << "Length of String = " << strlen(ary)<<endl;

return 0;

}
```

Output:

Length of String = 26

C++ String Functions

Function	Description
<code>int compare(const string& str)</code>	It is used to compare two string objects.
<code>int length()</code>	It is used to find the length of the string.
<code>void swap(string& str)</code>	It is used to swap the values of two string objects.
<code>string substr(int pos,int n)</code>	It creates a new string object of n characters.
<code>int size()</code>	It returns the length of the string in terms of bytes.
<code>void resize(int n)</code>	It is used to resize the length of the string

	up to n characters.
<code>string& replace(int pos,int len,string& str)</code>	It replaces portion of the string that begins at character position pos and spans len characters.
<code>string& append(const string& str)</code>	It adds new characters at the end of another string object.
<code>char& at(int pos)</code>	It is used to access an individual character at specified position pos.
<code>int find(string& str,int pos,int n)</code>	It is used to find the string specified in the parameter.
<code>int find_first_of(string& str,int pos,int n)</code>	It is used to find the first occurrence of the specified sequence.
<code>int find_first_not_of(string& str,int pos,int n)</code>	It is used to search the string for the first character that does not match with any of the characters specified in the string.
<code>int find_last_of(string& str,int pos,int n)</code>	It is used to search the string for the last character of specified sequence.

<code>int find_last_not_of(string& str, int pos)</code>	It searches for the last character that does not match with the specified sequence.
<code>string& insert()</code>	It inserts a new character before the character indicated by the position pos.
<code>int max_size()</code>	It finds the maximum length of the string.
<code>void push_back(char ch)</code>	It adds a new character ch at the end of the string.
<code>void pop_back()</code>	It removes a last character of the string.
<code>string& assign()</code>	It assigns new value to the string.
<code>int copy(string& str)</code>	It copies the contents of string into another.
<code>char& back()</code>	It returns the reference of last character.
<code>Iterator begin()</code>	It returns the reference of first character.
<code>int capacity()</code>	It returns the allocated space for the string.

<code>const_iterator cbegin()</code>	It points to the first element of the string.
<code>const_iterator cend()</code>	It points to the last element of the string.
<code>void clear()</code>	It removes all the elements from the string.
<code>const_reverse_iterator crbegin()</code>	It points to the last character of the string.
<code>const_char* data()</code>	It copies the characters of string into an array.
<code>bool empty()</code>	It checks whether the string is empty or not.
<code>string& erase()</code>	It removes the characters as specified.
<code>char& front()</code>	It returns a reference of the first character.
<code>string& operator+=()</code>	It appends a new character at the end of the string.
<code>string& operator=()</code>	It assigns a new value to the string.
<code>char operator[] (pos)</code>	It retrieves a character at specified position pos.

<code>intrfind()</code>	It searches for the last occurrence of the string.
<code>iteratorend()</code>	It references the last character of the string.
<code>reverse_iteratorrend()</code>	It points to the first character of the string.
<code>void shrink_to_fit()</code>	It reduces the capacity and makes it equal to the size of the string.
<code>char* c_str()</code>	It returns pointer to an array that contains null terminated sequence of characters.
<code>const_reverse_iteratorrend()</code>	It references the first character of the string.
<code>reverse_iteratorrbegin()</code>	It reference the last character of the string.
<code>void reserve(inrlen)</code>	It requests a change in capacity.
<code>allocator_type get_allocator();</code>	It returns the allocated object associated with the string.

C++ Exceptions

C++ Exception Handling

Exception Handling in C++ is a process to handle runtime errors. We perform exception handling so the normal flow of the application can be maintained even after runtime errors.

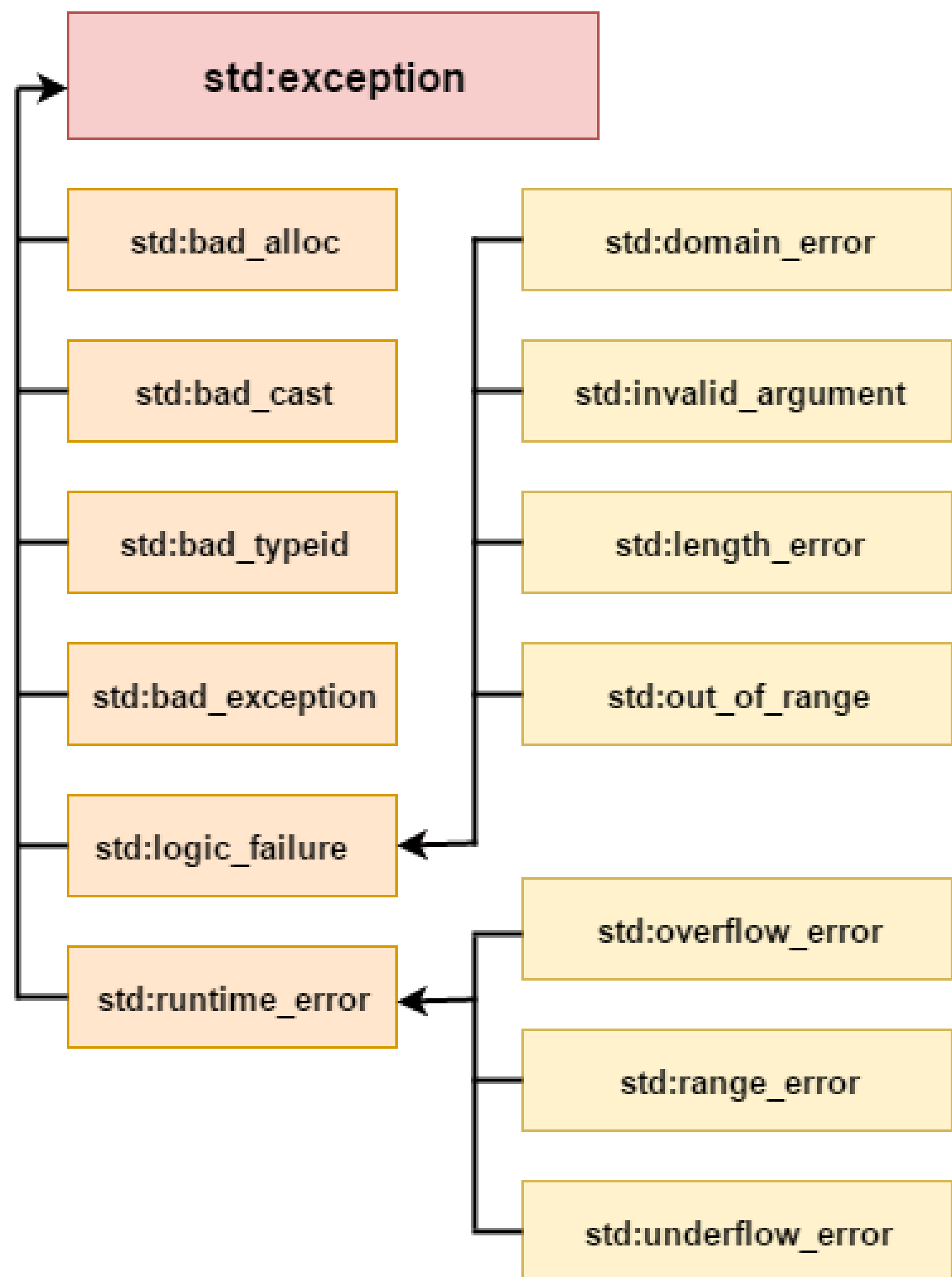
In C++, exception is an event or object which is thrown at runtime. All exceptions are derived from `std::exception` class. It is a runtime error which can be handled. If we don't handle the exception, it prints exception message and terminates the program.

Advantage

It maintains the normal flow of the application. In such case, rest of the code is executed even after exception.

C++ Exception Classes

In C++ standard exceptions are defined in `<exception>` class that we can use inside our programs. The arrangement of parent-child class hierarchy is shown below:



All the exception classes in C++ are derived from `std::exception` class. Let's see the list of C++ common exception classes.

Exception	Description
<code>std::exception</code>	It is an exception and parent class of all standard C++ exceptions.
<code>std::logic_failure</code>	It is an exception that can be detected by reading a code.

<code>std::runtime_error</code>	It is an exception that cannot be detected by reading a code.
<code>std::bad_exception</code>	It is used to handle the unexpected exceptions in a c++ program.
<code>std::bad_cast</code>	This exception is generally be thrown by <code>dynamic_cast</code> .
<code>std::bad_typeid</code>	This exception is generally be thrown by <code>typeid</code> .
<code>std::bad_alloc</code>	This exception is generally be thrown by <code>new</code> .

C++ Exception Handling Keywords

In C++, we use 3 keywords to perform exception handling:

- try
- catch, and
- throw

Moreover, we can create user-defined exception which we will learn in next chapters.

C++ try/catch

In C++ programming, exception handling is performed using try/catch statement. The C++ **try block** is used to place the code that may occur exception. The **catch block** is used to handle the exception.

C++ example without try/catch

```
#include <iostream>

using namespace std;

float division(int x, int y) {

    return (x/y);

}

int main () {

    int i = 50;

    int j = 0;

    float k = 0;

    k = division(i, j);

    cout << k << endl;

    return 0;

}
```


Output:

Floating point exception (core dumped)

C++ try/catch example

```
#include <iostream>

using namespace std;

float division(int x, int y) {

    if( y == 0 ) {

        throw "Attempted to divide by zero!";

    }

    return (x/y);

}

int main () {

    int i = 25;

    int j = 0;

    float k = 0;

    try {

        k = division(i, j);

        cout << k << endl;
```

```

    } catch (const char* e) {

        cerr << e << endl;

    }

    return 0;

}

```

Output:

Attempted to divide by zero!

C++ User-Defined Exceptions

The new exception can be defined by overriding and inheriting `exception` class functionality.

C++ user-defined exception example

Let's see the simple example of user-defined exception in which `std::exception` class is used to define the exception.

```

#include <iostream>

#include <exception>

using namespace std;

class MyException : public exception{

public:

    const char* what() const throw()

```

```

    {

        return "Attempted to divide by zero!\n";

    }

};

int main()

{

    try

    {

        int x, y;

        cout << "Enter the two numbers : \n";

        cin >> x >> y;

        if (y == 0)

        {

            MyException z;

            throw z;

        }

        else

        {

```



```

        cout << "x / y = " << x/y << endl;

    }

}

catch(exception& e)

{

    cout << e.what( );

}

}

```

Output:

Enter the two numbers :

10

2

x / y = 5

Output:

Enter the two numbers :

10

0

Attempted to divide by zero!

-->

Note: In above example what() is a public method provided by the exception class. It is used to return the cause of an exception.

C++ File & Streams

C++ Files and Streams

In C++ programming we are using the `iostream` standard library, it provides `cin` and `cout` methods for reading from input and writing to output respectively.

To read and write from a file we are using the standard C++ library called `fstream`. Let us see the data types define in `fstream` library is:

Data Type	Description
<code>fstream</code>	It is used to create files, write information to files, and read information from files.
<code>ifstream</code>	It is used to read information from files.
<code>ofstream</code>	It is used to create files and write information to the files.

C++ FileStream example: writing to a file

Let's see the simple example of writing to a text file `testout.txt` using C++ FileStream programming.


```

#include <iostream>

#include <fstream>

using namespace std;

int main ( ) {

    ofstream filestream( "testout.txt");

    if (filestream.is_open())

    {

        filestream << "Welcome to javaTpoint.\n";

        filestream << "C++ Tutorial.\n";

        filestream.close( );

    }

    else cout << "File opening is fail.";

    return 0;

}

```

Output:

The content of a text file `testout.txt` is set with the data:

Welcome to javaTpoint.

C++ Tutorial.

C++ FileStream example: reading from a file

Let's see the simple example of reading from a text file `testout.txt` using C++ FileStream programming.

```
#include <iostream>

#include <fstream>

using namespace std;

int main ( ) {

    string srg;

    ifstream filestream( "testout.txt" );

    if (filestream.is_open())

    {

        while ( getline (filestream,srg) )

        {

            cout << srg <<endl;

        }

        filestream.close( );

    }

    else {
```

```

        cout << "File opening is fail." << endl;

    }

    return 0;

}

```

Note: Before running the code a text file named as "testout.txt" is need to be created and the content of a text file is given below:

Welcome to javaTpoint C++ Tutorial.

Output:

```

Welcome to javaTpoint.

C++ Tutorial.

```

Let's see the simple example of writing the data to a text file **testout.txt** and then reading the data from the file using C++ FileStream programming.

```

#include <fstream>

#include <iostream>

using namespace std;

int main ( ) {

    char input[75];

    ofstream os;

    os.open("testout.txt");

```



```

cout << "Writing to a text file:" << endl;

cout << "Please Enter your name: ";

cin.getline(input, 100);

os << input << endl;

cout << "Please Enter your age: ";

cin >> input;

cin.ignore( );

os << input << endl;

os.close( );

ifstream is;

string line;

is.open("testout.txt");

cout << "Reading from a text file:" << endl;

while (getline (is,line))

{

cout << line << endl;

}

is.close( );

```

```
    return 0;

}
```

Output:

Writing to a text file:

Please Enter your name: Nakul Jain

Please Enter your age: 22

Reading from a text file: Nakul Jain

22

C++ getline()

The cin is an object which is used to take input from the user but does not allow to take the input in multiple lines. To accept the multiple lines, we use the getline() function. It is a pre-defined function defined in a `<string.h>` header file used to accept a line or a string from the input stream until the delimiting character is encountered.

Syntax of getline() function:

There are two ways of representing a function:

- The first way of declaring is to pass three parameters.

```
istream& getline( istream& is, string& str, char delim );
```

The above syntax contains three parameters, i.e., **is**, **str**, and **delim**.

Where,

is: It is an object of the `istream` class that defines from where to read the input stream.

str: It is a string object in which string is stored.

delim: It is the delimiting character.

Return value

This function returns the input stream object, which is passed as a parameter to the function.

- The second way of declaring is to pass two parameters.

```
istream& getline( istream& is, string& str );
```

The above syntax contains two parameters, i.e., **is** and **str**. This syntax is almost similar to the above syntax; the only difference is that it does not have any delimiting character.

Where,

is: It is an object of the `istream` class that defines from where to read the input stream.

str: It is a string object in which string is stored.

Return value

This function also returns the input stream, which is passed as a parameter to the function.

Let's understand through an example.

First, we will look at an example where we take the user input without using `getline()` function.


```
#include <iostream>

#include<string.h>

using namespace std;

int main()

{

string name; // variable declaration

std::cout << "Enter your name :" << std::endl;

cin>>name;

cout<<"\nHello " <<name;

return 0;

}
```

In the above code, we take the user input by using the statement `cin>>name`, i.e., we have not used the `getline()` function.

Output

Enter your name :

John Miller

Hello John

In the above output, we gave the name 'John Miller' as user input, but only 'John' was displayed. Therefore, we conclude that cin does not consider the character when the space character is encountered.

Let's resolve the above problem by using `getline()` function.

```
#include <iostream>

#include<string.h>

using namespace std;

int main()

{

    string name; // variable declaration.

    std::cout << "Enter your name :" << std::endl;

    getline(cin,name); // implementing a getline() function

    cout<<"\nHello " <<name;

    return 0;}
```

In the above code, we have used the `getline()` function to accept the character even when the space character is encountered.

Output

```
Enter your name :
```

```
John Miller
```


Hello John Miller

In the above output, we can observe that both the words, i.e., John and Miller, are displayed, which means that the `getline()` function considers the character after the space character also.

When we do not want to read the character after space then we use the following code:

```
#include <iostream>

#include<string.h>

using namespace std;

int main()

{

    string profile; // variable declaration

    std::cout << "Enter your profile : " << std::endl;

    getline(cin,profile,' '); // implementing getline() function with a delimiting character.

    cout<<"\nProfile is : "<<profile;

}
```

In the above code, we take the user input by using `getline()` function, but this time we also add the delimiting character(" ") in a third parameter. Here, delimiting character is a space character, means the character that appears after space will not be considered.

Output

Enter your profile :

Software Developer

Profile is: Software

Getline Character Array

We can also define the `getline()` function for character array, but its syntax is different from the previous one.

Syntax

```
istream& getline(char* , int size);
```

In the above syntax, there are two parameters; one is `char*`, and the other is `size`.

Where,

char*: It is a character pointer that points to the array.

Size: It acts as a delimiter that defines the size of the array means input cannot cross this size.

Let's understand through an example.

```
#include <iostream>
```

```
#include <string.h>
```

```
using namespace std;
```

```
int main()
```

```

{

char fruits[50]; // array declaration

cout<< "Enter your favorite fruit: ";

cin.getline(fruits, 50); // implementing getline() function

std::cout << "\nYour favorite fruit is :"<<fruits << std::endl;

return 0;

}

```

Output

Enter your favorite fruit: Watermelon

Your favorite fruit is: Watermelon

C++ Programs

Top 10+ C++ Programs

C++ programs are frequently asked in the interview. These programs can be asked from basics, array, string, pointer, linked list, file handling etc. Let's see the list of top c++ programs.

1) Fibonacci Series

Write a c++ program to print fibonacci series without using recursion and using recursion.

Input: 10

Output: 0112358132134

2) ~~Prime~~number

Write a c++ program to check prime number.

Input: 17

Output: not prime number

Input: 57

Output: prime number

3) ~~Palindrome~~number

Write a c++ program to check palindrome number.

Input: 121

Output: not palindrome number

Input: 113

Output: palindrome number

4) ~~Factorial~~

Write a c++ program to print factorial of a number.

Input: 5

Output: 120

Input: 6

Output: 720

5) Armstrong number

Write a c++ program to check armstrong number.

Input: 371

Output: armstrong

Input: 342

Output: not armstrong

6) Sum of Digits

Write a c++ program to print sum of digits.

Input: 23

Output: 5

Input: 624

Output: 12

7) Reverse Number

Write a c++ program to reverse given number.

Input: 234

Output: 432

8) Swap two numbers without using third variable

Write a c++ program to swap two numbers without using third variable.

Input: a=5 b=10

Output: a=10 b=5

9) Matrix Multiplication

Write a c++ program to print multiplication of 2 matrices.

Input:

first matrix elements:

1 2 3

1 2 3

1 2 3

second matrix elements

1 1 1

2 1 2

3 2 1

Output:

multiplication of the matrix:

14 9 8

14 9 8

14 9 8

10) Decimal to Binary

Write a c++ program to convert decimal number to binary.

Input: 9

Output: 1001

Input: 20

Output: 10100

11) Alphabet Triangle

Write a c++ program to print alphabet triangle.

Output:

```

  A
 ABA
ABCBA
ABCD CBA
ABCDEDCBA
```

12) Number Triangle

Write a c++ program to print number triangle.

Input: 7

Output:

```

enter the range= 6
```

```

  1
```

121
12321
1234321
123454321
12345654321

13) Fibonacci Triangle

Write a `c++` program to generate fibonacci triangle.

Input: 5

Output:

1
1 1
1 1 2
1 1 2 3
1 1 2 3 5

14) Number in Characters

Write a `c++` program to convert number in characters.

Input: 74254

Output: Seven Four Two Five Four

Input: 203

Output: two zero three

Fibonacci Series in C++

Fibonacci Series in C++: In case of fibonacci series, next number is the sum of previous two numbers for example 0, 1, 1, 2, 3, 5, 8, 13, 21 etc. The first two numbers of fibonacci series are 0 and 1.

There are two ways to write the fibonacci series program:

- Fibonacci Series without recursion
- Fibonacci Series using recursion

Fibonacci Series in C++ without Recursion

Let's see the fibonacci series program in C++ without recursion.

```
#include <iostream>

using namespace std;

int main() {

    int n1=0,n2=1,n3,i,number;

    cout<<"Enter the number of elements: ";

    cin>>number;

    cout<<n1<<" "<<n2<<" "; //printing 0 and 1

    for(i=2;i<number;++i) //loop starts from 2 because 0 and 1 are already printed
```

```

{

n3=n1+n2;

cout<<n3<<" ";

n1=n2;

n2=n3;

}

return 0;

}

```

Output:

Enter the number of elements: 10

0 1 1 2 3 5 8 13 21 34

Fibonacci series using recursion in C++

Let's see the fibonacci series program in C++ using recursion.

```

#include<iostream>

using namespace std;

void printFibonacci(int n){

    static int n1=0, n2=1, n3;

    if(n>0){

```

```

        n3 = n1 + n2;

        n1 = n2;

        n2 = n3;

    cout<<n3<<" ";

        printFibonacci(n-1);

    }

}

int main(){

    int n;

    cout<<"Enter the number of elements: ";

    cin>>n;

    cout<<"Fibonacci Series: ";

    cout<<"0 "<<"1 ";

    printFibonacci(n-2); //n-2 because 2 numbers are already printed

    return 0;

}

```

Output:

Enter the number of elements: 15

Fibonacci Series: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Prime Number Program in C++

Prime number is a number that is greater than 1 and divided by 1 or itself. In other words, prime numbers can't be divided by other numbers than itself or 1. For example 2, 3, 5, 7, 11, 13, 17, 19, 23....are the prime numbers.

Let's see the prime number program in C++. In this C++ program, we will take an input from the user and check whether the number is prime or not.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int n, i, m=0, flag=0;
```

```
    cout << "Enter the Number to check Prime: ";
```

```
    cin >> n;
```

```
    m=n/2;
```

```
    for(i = 2; i <= m; i++)
```

```
{
```

```
        if(n % i == 0)
```

```
{
```

```

        cout<<"Number is not Prime."<<endl;

        flag=1;

        break;

    }

}

if (flag==0)

    cout << "Number is Prime."<<endl;

return 0;

}

```

Output:

Enter the Number to check Prime: 17

Number is Prime.

Enter the Number to check Prime: 57

Number is not Prime.

Palindrome program in C++

A **palindrome number** is a number that is same after reverse. For example 121, 34543, 343, 131, 48984 are the palindrome numbers.

Palindrome number algorithm

- Get the number from user
- Hold the number in temporary variable
- Reverse the number
- Compare the temporary number with reversed number
- If both numbers are same, print palindrome number
- Else print not palindrome number

Let's see the palindrome program in C++ . In this program, we will get an input from the user and check whether number is palindrome or not.

```
#include <iostream>

using namespace std;

int main( )

{

    int n,r,sum=0,temp;

    cout<<"Enter the Number=";

    cin>>n;

    temp=n;

    while(n>0)

    {
```



```

r=n%10;

sum=(sum*10)+r;

n=n/10;

}

if(temp==sum)

cout<<"Number is Palindrome.";

else

cout<<"Number is not Palindrome.";

return 0;

}

```

Output:

Enter the Number=121

Number is Palindrome.

Enter the number=113

Number is not Palindrome.

Factorial program in C++

Factorial Program in C++: Factorial of n is the product of all positive descending integers.

Factorial of n is denoted by n!. For example:

$$1. \quad 4! = 4 \times 3 \times 2 \times 1 = 24$$

$$2. 6! = 6 * 5 * 4 * 3 * 2 * 1 = 720$$

Here, 4! is pronounced as "4 factorial", it is also called "4 bang" or "4 shriek".

The factorial is normally used in Combinations and Permutations (mathematics).

There are many ways to write the factorial program in C++ language. Let's see the 2 ways to write the factorial program.

- Factorial Program using loop
- Factorial Program using recursion

Factorial Program using Loop

Let's see the factorial Program in C++ using loop.

```
#include <iostream>

using namespace std;

int main()
{
    int i, fact=1, number;

    cout<<"Enter any Number: ";

    cin>>number;

    for(i=1; i<=number; i++){

        fact=fact*i;
```



```

    }

    cout<<"Factorial of " <<number<<" is: " <<fact<<endl;

    return 0;

}

```

Output:

Enter any Number: 5

Factorial of 5 is: 120

Factorial Program using Recursion

Let's see the factorial program in C++ using recursion.

```

#include<iostream>

using namespace std;

int main()

{

    int factorial(int);

    int fact,value;

    cout<<"Enter any number: ";

    cin>>value;

    fact=factorial(value);

```

```
cout<<"Factorial of a number is: "<<fact<<endl;
```

```
return 0;
```

```
}
```

```
int factorial(int n)
```

```
{
```

```
if(n<0)
```

```
return(-1); /*Wrong value*/
```

```
if(n==0)
```

```
return(1); /*Terminating condition*/
```

```
else
```

```
{
```

```
return(n*factorial(n-1));
```

```
}
```

```
}
```

Output:

```
Enter any number: 6
```

```
Factorial of a number is: 720
```

Armstrong Number in C++

Before going to write the C++ program to check whether the number is Armstrong or not, let's understand what is Armstrong number.

Armstrong number is a number that is equal to the sum of cubes of its digits. For example 0, 1, 153, 370, 371 and 407 are the Armstrong numbers.

Let's try to understand why **371** is an Armstrong number.

$$371 = (3*3*3) + (7*7*7) + (1*1*1)$$

where:

$$(3*3*3)=27$$

$$(7*7*7)=343$$

$$(1*1*1)=1$$

So:

$$27+343+1=371$$

Let's see the C++ program to check Armstrong Number.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```



```

int n,r,sum=0,temp;

cout<<"Enter the Number= ";

cin>>n;

temp=n;

while(n>0)

{

r=n%10;

sum=sum+(r*r*r);

n=n/10;

}

if(temp==sum)

cout<<"Armstrong Number."<<endl;

else

cout<<"Not Armstrong Number."<<endl;

return 0;

}

```

Output:

Enter the Number= 371

Armstrong Number .

Enter the Number= 342

Not Armstrong Number .

Sum of digits program in C++

We can write the sum of digits program in C++ language by the help of loop and mathematical operation only.

Sum of digits algorithm

To get sum of each digit by C++ program, use the following algorithm:

- **Step 1:** Get number by user
- **Step 2:** Get the modulus/remainder of the number
- **Step 3:** sum the remainder of the number
- **Step 4:** Divide the number by 10
- **Step 5:** Repeat the step 2 while number is greater than 0.

Let's see the sum of digits program in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int n,sum=0,m;

cout<<"Enter a number: ";

cin>>n;

while(n>0)

{

m=n%10;

sum=sum+m;

n=n/10;

}

cout<<"Sum is= "<<sum<<endl;

return 0;

}
```

Output:

Enter a number: 23

Sum is= 5

Enter a number: 624

Sum is= 12

C++ Program to reverse number

We can reverse a number in C++ using loop and arithmetic operators. In this program, we are getting number as input from the user and reversing that number.

Let's see a simple C++ example to reverse a given number.

```
#include <iostream>

using namespace std;

int main()

{

    int n, reverse=0, rem;

    cout<<"Enter a number: ";

    cin>>n;

    while(n!=0)

    {

        rem=n%10;

        reverse=reverse*10+rem;

        n/=10;

    }

    cout<<"Reversed Number: "<<reverse<<endl;
```

```
return 0;

}
```

Output:

Enter a number: 234

Reversed Number: 432

C++ Program to swap two numbers without third variable

We can swap two numbers without using third variable. There are two common ways to swap two numbers without using third variable:

1. By + and -
2. By * and /

Program 1: Using * and /

Let's see a simple C++ example to swap two numbers without using third variable.

```
#include <iostream>

using namespace std;

int main()

{

    int a=5, b=10;

    cout<<"Before swap a= "<<a<<" b= "<<b<<endl;
```



```
a=a*b; //a=50 (5*10)
```

```
b=a/b; //b=5 (50/10)
```

```
a=a/b; //a=10 (50/5)
```

```
cout<<"After swap a= "<<a<<" b= "<<b<<endl;
```

```
return 0;
```

```
}
```

Output:

Before swap a= 5 b= 10

After swap a= 10 b= 5

Program 2: Using + and -

Let's see another example to swap two numbers using + and -.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a=5, b=10;
```

```
cout<<"Before swap a= "<<a<<" b= "<<b<<endl;
```

```
a=a+b; //a=15 (5+10)
```

```
b=a-b; //b=5 (15-10)
```

```
a=a-b; //a=10 (15-5)
```

```
cout<<"After swap a= "<<a<<" b= "<<b<<endl;
```

```
return 0;
```

```
}
```

Output:

Before swap a= 5 b= 10

After swap a= 10 b= 5

Matrix multiplication in C++

We can add, subtract, multiply and divide 2 matrices. To do so, we are taking input from the user for row number, column number, first matrix elements and second matrix elements.

Then we are performing multiplication on the matrices entered by the user.

In matrix multiplication first matrix one row element is multiplied by second matrix all column elements.

Let's try to understand the matrix multiplication of 3×3 and 3×3 matrices by the figure given below:

$$\text{Matrix 1} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\} \quad \text{Matrix 2} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\}$$

$$\begin{array}{l} \text{Matrix 1} \\ * \\ \text{Matrix 2} \end{array} \left\{ \begin{array}{ccc} 1*1+1*2+1*3 & 1*1+1*2+1*3 & 1*1+1*2+1*3 \\ 2*1+2*2+2*3 & 2*1+2*2+2*3 & 2*1+2*2+2*3 \\ 3*1+3*2+3*3 & 3*1+3*2+3*3 & 3*1+3*2+3*3 \end{array} \right\}$$

$$\begin{array}{l} \text{Matrix 1} \\ * \\ \text{Matrix 2} \end{array} \left\{ \begin{array}{ccc} 6 & 6 & 6 \\ 12 & 12 & 12 \\ 18 & 18 & 18 \end{array} \right\}$$

JavaTpoint

Let's see the program of matrix multiplication in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a[10][10],b[10][10],mul[10][10],r,c,i,j,k;
```

```
cout<<"enter the number of row=";
```

```
cin>>r;
```

```
cout<<"enter the number of column=";
```

```
cin>>c;
```



```
cout<<"enter the first matrix element=\n";
```

```
for(i=0;i<r;i++)
```

```
{
```

```
for(j=0;j<c;j++)
```

```
{
```

```
cin>>a[i][j];
```

```
}
```

```
}
```

```
cout<<"enter the second matrix element=\n";
```

```
for(i=0;i<r;i++)
```

```
{
```

```
for(j=0;j<c;j++)
```

```
{
```

```
cin>>b[i][j];
```

```
}
```

```
}
```

```
cout<<"multiply of the matrix=\n";
```

```
for(i=0;i<r;i++)
```

```

{

for(j=0;j<c;j++)

{

mul[i][j]=0;

for(k=0;k<c;k++)

{

mul[i][j]+=a[i][k]*b[k][j];

}

}

}

//for printing result

for(i=0;i<r;i++)

{

for(j=0;j<c;j++)

{

cout<<mul[i][j]<<" ";

}

cout<<"\n";

```

```
}  
  
return 0;  
  
}
```

Output:

```
enter the number of row=3  
  
enter the number of column=3  
  
enter the first matrix element=  
  
1 2 3  
  
1 2 3  
  
1 2 3  
  
enter the second matrix element=  
  
1 1 1  
  
2 1 2  
  
3 2 1  
  
multiply of the matrix=  
  
14 9 8  
  
14 9 8  
  
14 9 8
```

C++ Program to convert Decimal to Binary

We can convert any decimal number (base-10 (0 to 9)) into binary number (base-2 (0 or 1)) by C++ program.

Decimal Number

Decimal number is a base 10 number because it ranges from 0 to 9, there are total 10 digits between 0 to 9. Any combination of digits is decimal number such as 223, 585, 192, 0, 7 etc.

Binary Number

Binary number is a base 2 number because it is either 0 or 1. Any combination of 0 and 1 is binary number such as 1001, 101, 1111, 101010 etc.

Let's see the some binary numbers for the decimal number.

Decimal	Binary
1	0
2	10
3	11
4	100
5	101
6	110

7	111
8	1000
9	1001
10	1010

Decimal to Binary Conversion Algorithm

Step 1: Divide the number by 2 through % (modulus operator) and store the remainder in array

Step 2: Divide the number by 2 through / (division operator)

Step 3: Repeat the step 2 until the number is greater than zero

Let's see the C++ example to convert decimal to binary.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a[10], n, i;
```

```
cout<<"Enter the number to convert: ";
```



```

cin>>n;

for(i=0; n>0; i++)

{

a[i]=n%2;

n= n/2;

}

cout<<"Binary of the given number= ";

for(i=i-1 ;i>=0 ;i--)

{

cout<<a[i];

}

}

```

Output:

Enter the number to convert: 9

Binary of the given number= 1001

C++ Program to Convert Number in Characters

In C++ language, we can easily convert number in characters by the help of loop and switch case. In this program, we are taking input from the user and iterating this number until it is 0.

While iteration, we are dividing it by 10 and the remainder is passed in switch case to get the word for the number.

Let's see the C++ program to convert number in characters.

```
#include <iostream>

using namespace std;

int main()

{

    long int n,sum=0,r;

    cout<<"Enter the Number= ";

    cin>>n;

    while(n>0)

    {

        r=n%10;

        sum=sum*10+r;

        n=n/10;

    }

    n=sum;

    while(n>0)
```

```
{  
  
r=n%10;  
  
switch(r)  
  
{  
  
case 1:  
  
cout<<"one ";  
  
break;  
  
case 2:  
  
cout<<"two ";  
  
break;  
  
case 3:  
  
cout<<"three ";  
  
break;  
  
case 4:  
  
cout<<"four ";  
  
break;  
  
case 5:  
  
cout<<"five ";
```



```
break;
```

```
case 6:
```

```
cout<<"six ";
```

```
break;
```

```
case 7:
```

```
cout<<"seven ";
```

```
break;
```

```
case 8:
```

```
cout<<"eight ";
```

```
break;
```

```
case 9:
```

```
cout<<"nine ";
```

```
break;
```

```
case 0:
```

```
cout<<"zero ";
```

```
break;
```

```
default:
```

```
cout<<"ttt ";
```

```
break;
```

```
}
```

```
n=n/10;
```

```
}
```

```
}
```

Output:

```
Enter the Number= 74254
```

```
seven four two five four
```

C++ Program to Print Alphabet Triangle

There are different triangles that can be printed. Triangles can be generated by alphabets or numbers. In this C++ program, we are going to print alphabet triangles.

Let's see the C++ example to print alphabet triangle.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
char ch='A';
```

```
int i, j, k, m;
```

```

for(i=1;i<=5;i++)

{

    for(j=5;j>=i;j--)

        cout<<" ";

    for(k=1;k<=i;k++)

        cout<<ch++;

    ch--;

    for(m=1;m<i;m++)

        cout<<--ch;

    cout<<"\n";

    ch='A';

}

return 0;

}

```

Output:

```

A
ABA
ABCBA
ABCD CBA

```


ABCDEDCBA

C++ Program to print Number Triangle

Like alphabet triangle, we can write the C++ program to print the number triangle. The number triangle can be printed in different ways.

Let's see the C++ example to print number triangle.

```
#include <iostream>

using namespace std;

int main()

{

    int i,j,k,l,n;

    cout<<"Enter the Range=";

    cin>>n;

    for(i=1;i<=n;i++)

    {

        for(j=1;j<=n-i;j++)

        {

            cout<<" ";

        }

    }
```

```

for(k=1;k<=i;k++)

{

cout<<k;

}

for(l=i-1;l>=1;l--)

{

cout<<l;

}

cout<<"\n";

}

return 0;

}

```

Output:

Enter the Range=5

1

121

12321

1234321

123454321

Enter the Range=6

```
1
121
2321
1234321
123454321
12345654321
```

C++ Program to generate Fibonacci Triangle

In this program, we are getting input from the user for the limit for fibonacci triangle, and printing the fibonacci series for the given number of times (limit).

Let's see the C++ example to generate fibonacci triangle.

```
#include <iostream>

using namespace std;

int main()
{
    int a=0,b=1,i,c,n,j;

    cout<<"Enter the limit: ";

    cin>>n;

    for(i=1; i<=n; i++)
```

```

    {

        a=0;

        b=1;

        cout<<b<<"\t";

        for(j=1; j<i; j++)

        {

            c=a+b;

            cout<<c<<"\t";

            a=b;

            b=c;

        }

        cout<<"\n";

    }

    return 0;

}

```

Output:

Enter the limit: 10

1

1	1								
1	1	2							
1	1	2	3						
1	1	2	3	5					
1	1	2	3	5	8				
1	1	2	3	5	8	13			
1	1	2	3	5	8	13	21		
1	1	2	3	5	8	13	21		
34									
1	1	2	3	5	8	13	21		
34	55								

Char array to string in C++

"Char" data type or a character data type is used to store letters, unlike numbers and integers, which are stored in integer and floating-point or true-false value in Booleans.

Characters are integer type in nature, their size is 1-byte and printable characters are (space), !, ", #, \$, %, &, ', (,), *, +, ,, -, ., /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, :, ;, <, =, >, ?, @, A, a, B, b, C, c, D, d, E, e, F, f, G, g, H, h, I, i, J, j, K, k, L, l, M, m, N, n, O, o, P, p, Q, q, R, r, S, s, T, t, U, u, V, v, W, w, X, x, Y, y, Z, z, [, \,], ^, _, `, {, |, }, ~ and DEL (delete).

We can initialize char variables using -

`char ch2{'a'};` To print character "a".

`char ch1{97};` To print the value at code 97.

`char ch{'5'};` To print number character "5".

C++ provides us with the following techniques to convert a char array to a string:

- Using 'c_str()' and 'strcpy()' function
- using a 'for' loop
- 'while' loop,
- '=' operator from the string class
- using custom function.

Syntax for using 'c_str()' and 'strcpy()' function is given below:

`"string-name.c_str();"`

where, `c_str()` converts the contents of a string as C-style, non-terminated string. It gives direct access to the internal string buffer.

We can convert character to a string using 'for' a loop by-

First, declaring the Character Array and then assigning the size of the Array. Then, we declare two variables, one string type, and another int type. After this, we can use the 'for' loop by assigning 0 to the int variable where the int variable will have less value than `array_size` and we increase int variable value by one at every iteration. We have to store the value in the string variable on every iteration before displaying the string variable.

Code:-

```
#include <iostream>
```

```

using namespace std;

int main()

{

    char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};

    int array_size = sizeof(char_array) /

    sizeof(char);

    string j = "";

    int i;

    for(i = 0; i < array_size; i++)

    {

        /** retrieving and merging the value of character array on position 'i' */

        j = j + char_array[i];

    }

    cout << j << endl;

}

```

Output:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
6     int array_size = sizeof(char_array) /
7     sizeof(char);
8     string j = "";
9     int i;
10    for(i = 0; i < array_size; i++)
11    {
12        /** retrieving and merging the value of character array on
13        position 'i'*/
14        j = j + char_array[i];
15    }
16    cout << j << endl;
17 }
18
```

Output

```
g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp
/tmp/9fX07E95K4.o
JAVATPOINT
```

We can convert char to a string using 'while' loop by -

First declaring the Character Array and then assigning the size of the Array. Then, we declare two variables, one string type, and another int type with value 0. We use 'while' loop to check int variable less than array_size on every iteration and store the value in a string variable before displaying the string variable.

Code:-

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
```



```

int array_size = sizeof(char_array) / sizeof(char);

string j = "";

int i = 0;

while(i < array_size)
{
    /** retrieving and merging the value of character array on position `i` */

    j = j + char_array[i];

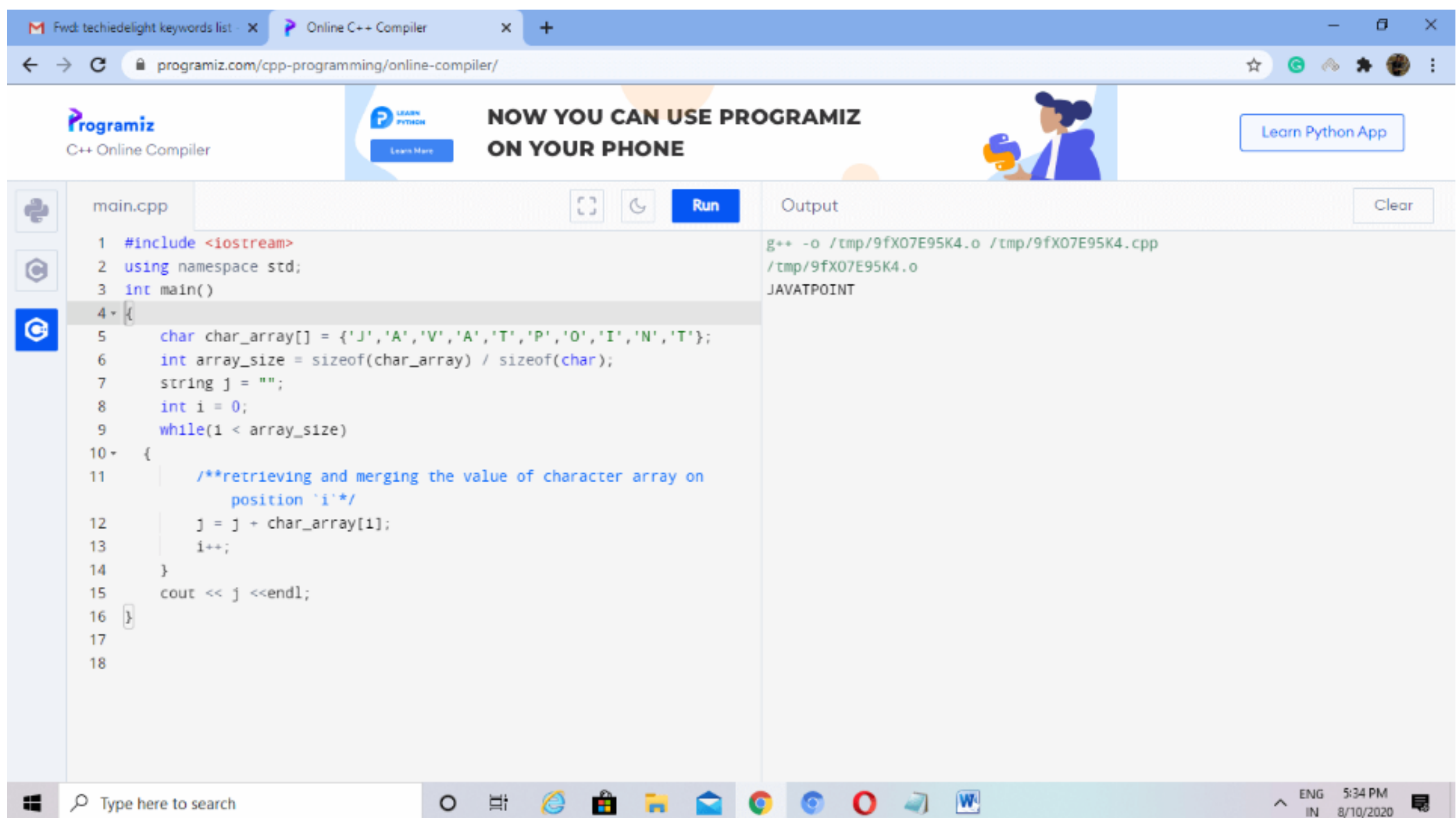
    i++;
}

cout << j << endl;

}

```

Output:



The screenshot shows the Programiz C++ Online Compiler interface. The code editor on the left contains the following C++ code:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
6     int array_size = sizeof(char_array) / sizeof(char);
7     string j = "";
8     int i = 0;
9     while(i < array_size)
10    {
11        /**retrieving and merging the value of character array on
12         position 'i'*/
13        j = j + char_array[i];
14        i++;
15    }
16    cout << j << endl;
17 }
18
```

The output panel on the right shows the command used to compile the code: `g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp` and the output: `JAVATPOINT`.

To convert character to a string using `std::string` constructor, we simply pass the array to string constructor.

Code:-

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
```

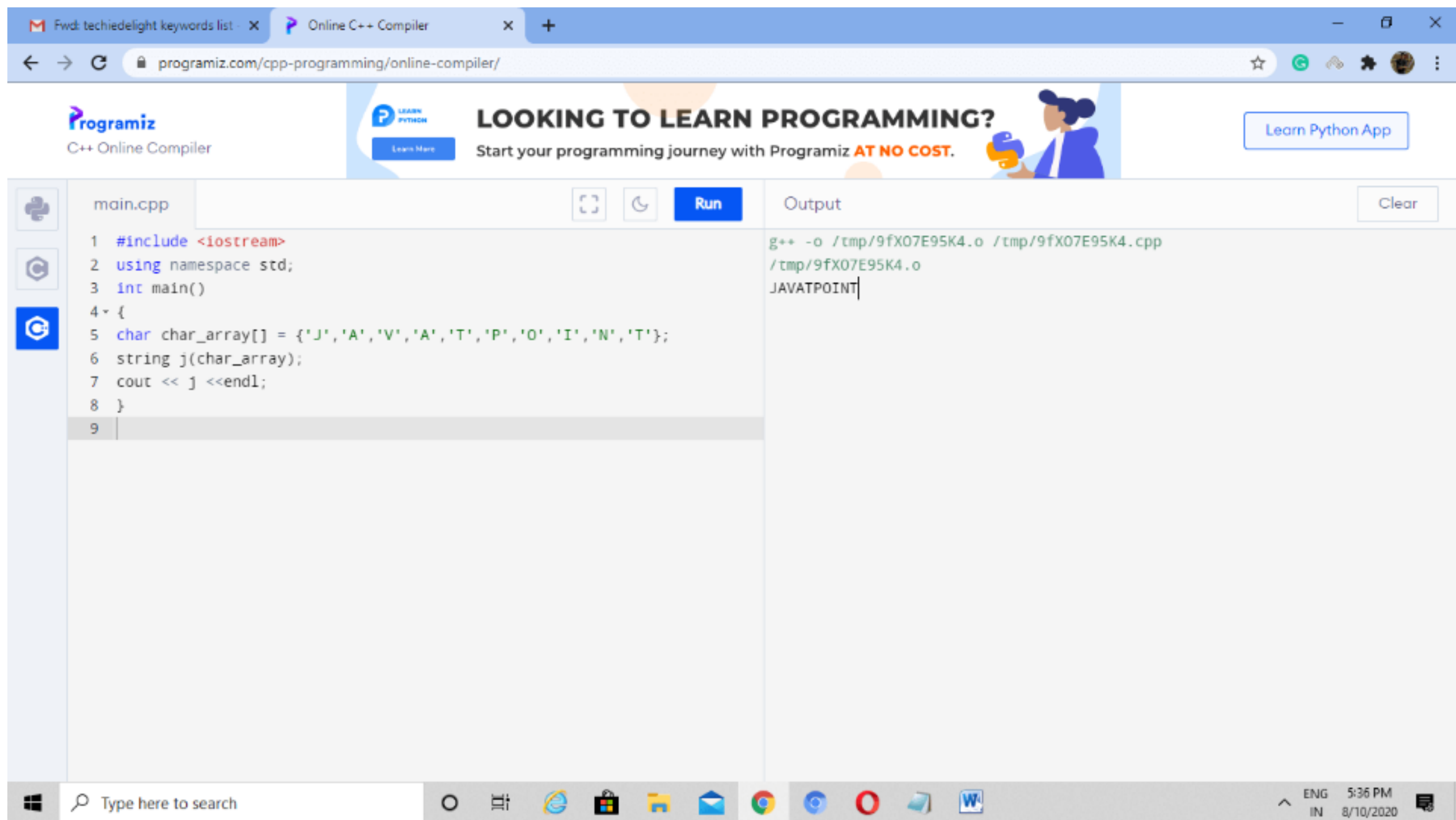
```
string j(char_array);
```

```
cout << j << endl;
```



```
}
```

Output:



The screenshot shows a web browser window with the URL `programiz.com/cpp-programming/online-compiler/`. The page features a banner for Programiz with the text "LOOKING TO LEARN PROGRAMMING?" and "Start your programming journey with Programiz AT NO COST." Below the banner, there is a code editor with a file named `main.cpp`. The code in the editor is as follows:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
6     string j(char_array);
7     cout << j << endl;
8 }
9
```

To the right of the code editor is an "Output" window. It shows the command `g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp` and the output `JAVATPOINT`. The browser's taskbar at the bottom shows the Windows logo, a search bar, and several application icons. The system clock in the bottom right corner indicates the time is 5:36 PM on 8/10/2020.

To convert a character array into a string using the '=' operator and string class, we have to pass character array to string variable.

Code:-

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
```

```

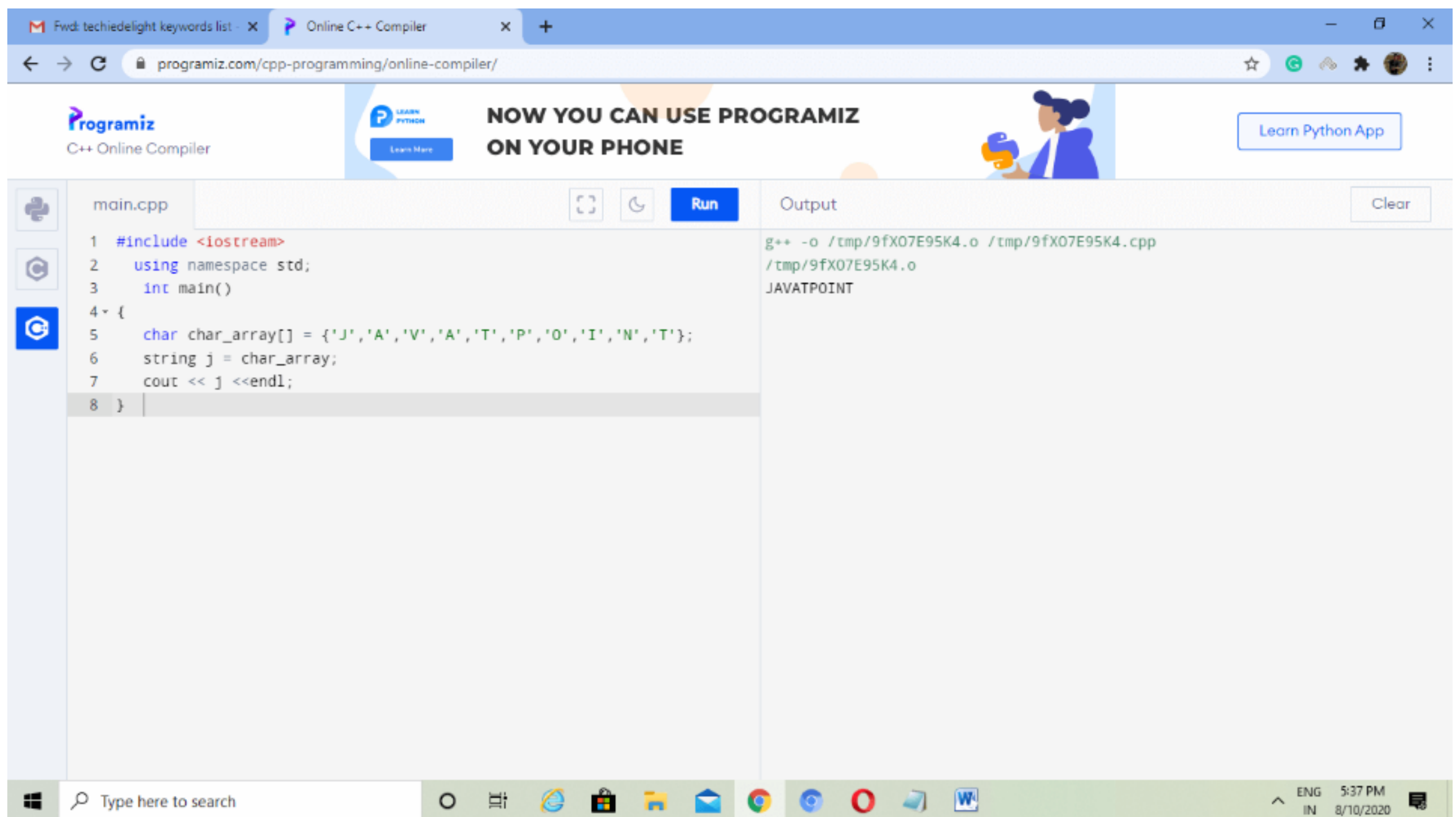
string j = char_array;

cout << j << endl;

}

```

Output:



The screenshot shows the Programiz Online C++ Compiler interface. The code editor on the left contains the following C++ code:

```

1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
6     string j = char_array;
7     cout << j << endl;
8 }

```

The output window on the right shows the compilation and execution results:

```

g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp
/tmp/9fX07E95K4.o
JAVATPOINT

```

The browser's address bar shows the URL: programiz.com/cpp-programming/online-compiler/. The Windows taskbar at the bottom indicates the system time is 5:37 PM on 8/10/2020.

To convert character to a string using custom functions, we have to create a custom function with two parameters. Inside the custom function, we have to declare two variables string and integer. Then we use the 'for' loop, where we assign 0 to an int variable, int variable's size to be less than array_size, and int variable's value increasing by one at each iteration. The function will return the string. For the primary function, we declare the character array and its size, then we pass character array and its size to the custom function. At last, we print the string variable which stores the returned value of the custom function.

Code-

```
#include <iostream>

using namespace std;

string charToString(char* arr_char, int arr_size)

{

    int i;

    string j = "";

    for(i = 0; i < arr_size; i++)

    {

        j = j + arr_char[i];

    }

    return j;

}

int main() {

    char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};

    int array_size = sizeof(char_array) / sizeof(char);

    string jtp = "";

    jtp = charToString(char_array, array_size);
```



```

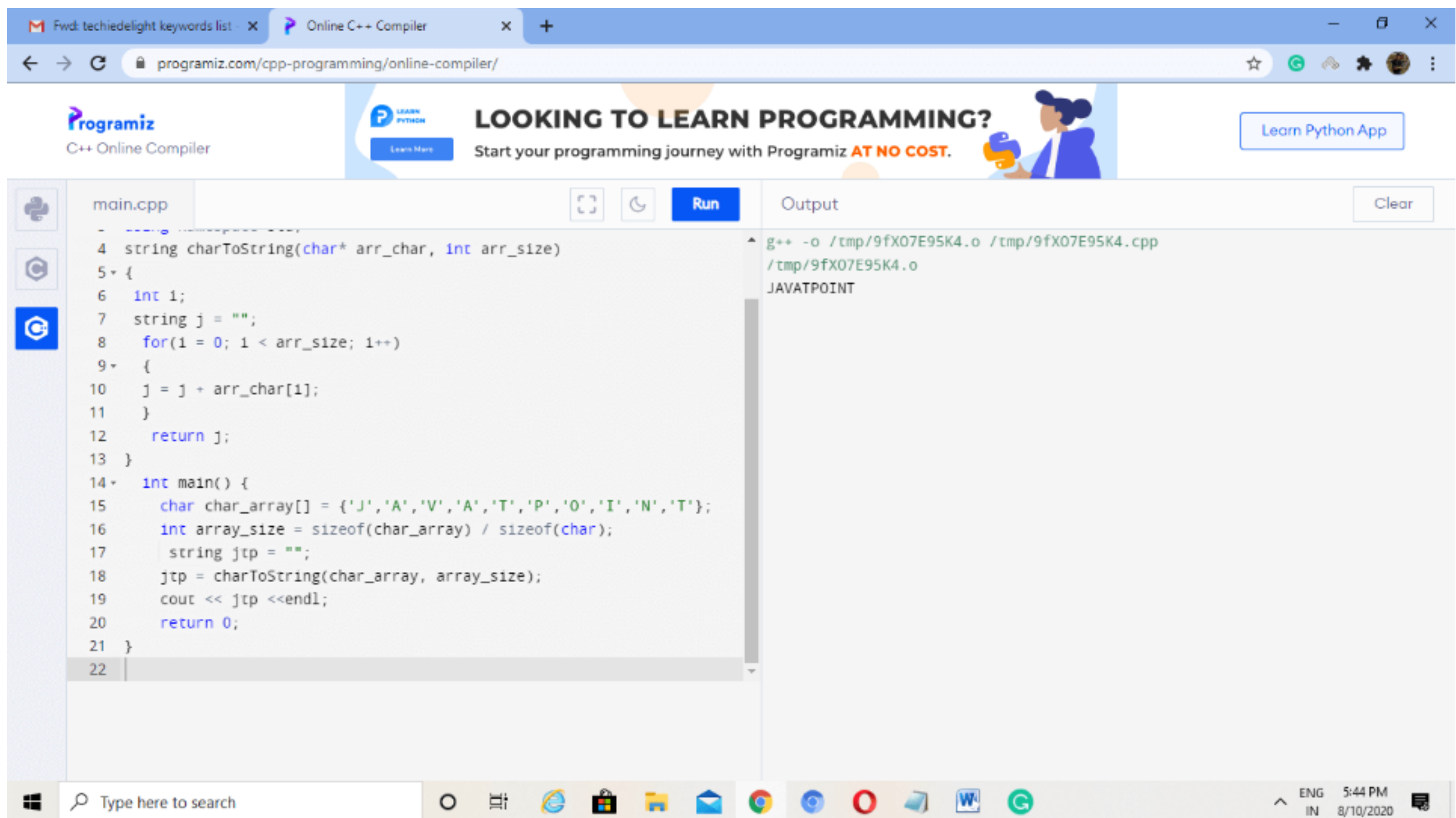
cout << jtp << endl;

return 0;

}

```

Output:



The screenshot shows the Programiz Online C++ Compiler interface. The code in the editor is as follows:

```

main.cpp
4 string charToString(char* arr_char, int arr_size)
5 {
6     int i;
7     string j = "";
8     for(i = 0; i < arr_size; i++)
9     {
10        j = j + arr_char[i];
11    }
12    return j;
13 }
14 int main() {
15     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
16     int array_size = sizeof(char_array) / sizeof(char);
17     string jtp = "";
18     jtp = charToString(char_array, array_size);
19     cout << jtp << endl;
20     return 0;
21 }
22

```

The output of the program is displayed in the Output panel:

```

g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp
/tmp/9fX07E95K4.o
JAVATPOINT

```

The last method to convert char to string is achieved by using `std::stringstream`. We use this function to insert the input character into a buffer and then take the character from the buffer as a string using `std::string`.

CODE-

```

#include <iostream>

#include <sstream>

int main()

```

```

{

char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};

std::string jtp;

std::stringstream strStream;

strStream << char_array;

strStream >> jtp;

std::cout << jtp;

return 0;

}

```

Output:

The screenshot shows the Programiz Online C++ Compiler interface. The code editor on the left contains the following C++ code:

```

1 #include <iostream>
2 #include <sstream>
3 int main()
4 {
5     char char_array[] = {'J','A','V','A','T','P','O','I','N','T'};
6     std::string jtp;
7     std::stringstream strStream;
8     strStream << char_array;
9     strStream >> jtp;
10    std::cout << jtp;
11    return 0;
12 }
13

```

The output window on the right shows the compilation command and the resulting output:

```

g++ -o /tmp/9fX07E95K4.o /tmp/9fX07E95K4.cpp
/tmp/9fX07E95K4.o
JAVATPOINT

```

The browser's address bar shows the URL: programiz.com/cpp-programming/online-compiler/. The Windows taskbar at the bottom shows the time as 5:45 PM on 8/10/2020.

Calculator Program in C++

A calculator is a portable device that helps to perform simple mathematical calculations in our daily lives such as **addition, subtraction, division, multiplication**, etc. Some of the scientific calculators are used to perform complex calculation more easily like square root, function, exponential operations, logarithm, trigonometric function and hyperbolic function, etc. In this section, we will create **calculator program in C++ using function and do-while loop**.



Using Function

Lets' create a calculator program in C++ using the **function** and **Switch** statement.

```
#include<iostream.h>
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<math.h>
```

```
#include<stdlib.h>
```

```
void add();
```

```
void sub();
```

```
void multi();
```

```
void division();
```

```
void sqr();
```

```
void srt();
```

```
void exit();
```

```
void main()
```

```
{
```

```
clrscr();
```

```
int opr;
```

```
// display different operation of the calculator
```

```
do
```

```
{
```

```
cout << "Select any operation from the C++ Calculator"
```

```
"\n1 = Addition"
```

```
"\n2 = Subtraction"
```

```
"\n3 = Multiplication"
```

```
"\n4 = Division"
```

```
"\n5 = Square"
```

```
"\n6 = Square Root"
```

```
"\n7 = Exit"
```

```
"\n\n Make a choice: ";
```

```
cin >> opr;
```

```
switch (opr)
```

```
{
```

```
case 1:
```

```
add(); // call add() function to find the Addition
```

```
break;
```

```
case 2:
```

```
sub(); // call sub() function to find the subtraction
```

```
break;
```


case 3:

multi(); // call multi() function to find the multiplication

break;

case 4:

division(); // call division() function to find the division

break;

case 5:

sqr(); // call sqr() function to find the square of a number

break;

case 6:

srt(); // call srt() function to find the Square Root of the given number

break;

case 7:

exit(0); // terminate the program

break;

default:

cout << "Something is wrong..!!";

break;

```
}
```

```
cout <<" \n-----\n";
```

```
}while(opr != 7);
```

```
getch();
```

```
}
```

```
void add()
```

```
{
```

```
int n, sum = 0, i, number;
```

```
cout <<"How many numbers you want to add: ";
```

```
cin >> n;
```

```
cout << "Please enter the number one by one: \n";
```

```
for (i = 1; i <= n; i++)
```

```
{
```

```
cin >> number;
```

```
sum = sum + number;
```

```
}
```

```
cout << "\n Sum of the numbers = "<< sum;
```

```
}
```

```
void sub()
```

```
{
```

```
int num1, num2, z;
```

```
cout << "\n Enter the First number = ";
```

```
cin >> num1;
```

```
cout << "\n Enter the Second number = ";
```

```
cin >> num2;
```

```
z = num1 - num2;
```

```
cout << "\n Subtraction of the number = " << z;
```

```
}
```

```
void multi()
```

```
{
```

```
int num1, num2, mul;
```

```
cout << "\n Enter the First number = ";
```

```
cin >> num1;
```

```
cout << "\n Enter the Second number = ";
```

```
cin >> num2;
```



```
mul = num1 * num2;
```

```
cout << "\n Multiplication of two numbers = " << mul;
```

```
}
```

```
void division()
```

```
{
```

```
int num1, num2, div = 0;
```

```
cout << " \n Enter the First number = ";
```

```
cin >> num1;
```

```
cout << " \n Enter the Second number = ";
```

```
cin >> num2;
```

```
while ( num2 == 0)
```

```
{
```

```
cout << " \n Divisor cannot be zero"
```

```
    "\n Please enter the divisor once again: ";
```

```
    cin >> num2;
```

```
}
```

```
div = num1 / num2;
```

```
cout << "\n Division of two numbers = " << div;
```

```
}
```

```
void sqr( )
```

```
{
```

```
int num1;
```

```
float sq;
```

```
cout << " \n Enter a number to find the Square: ";
```

```
cin >> num1;
```

```
sq = num1 * num1;
```

```
cout << " \n Square of " << num1 << " is : " << sq;
```

```
}
```

```
void srt( )
```

```
{
```

```
float q;
```

```
int num1;
```

```
cout << " \n Enter the number to find the Square Root:";
```

```
cin >> num1;
```

```
q = sqrt(num1);
```

```
cout << " \n Square Root of " << num1 << " is : " << q;
```

}

Output:

```
DOS BOX DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Select any operation from the C++ Calculator
1 = Addition
2 = Subtraction
3 = Multiplication
4 = Division
5 = Square
6 = Square Root
7 = Exit

Make a choice: 1
How many numbers you want to add: 5
Please enter the number one by one:
3
4
5
6
7

Sum of the numbers = 25
-----
Select any operation from the C++ Calculator
1 = Addition
2 = Subtraction
3 = Multiplication
4 = Division
5 = Square
6 = Square Root
7 = Exit

Make a choice: 6

Enter the number to find the Square Root:625

Square Root of 625 is : 25
-----
Select any operation from the C++ Calculator
1 = Addition
2 = Subtraction
3 = Multiplication
4 = Division
5 = Square
6 = Square Root
7 = Exit

Make a choice:
```

Using do-while Loop

Write a Calculator Program in the C++ using the **dowhile** and **Switch** Statement.

```
#include<iostream.h>
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
int opr;
```

```
int num1, num2, x;
```

```
// display different operation of the calculator
```

```
do
```

```
{
```

```
cout << "Select an operation to perform a simple calculation in C++ Calculator"
```

```
"\n1 = Addition"
```

```
"\n2 = Subtraction"
```

```
"\n3 = Multiplication"
```

```
"\n4 = Division"
```

```
"\n5 = Square"
```

```
"\n6 = Exit"
```

```
"\n\n Make a choice: ";
```

```
cin >> opr;
```

```
switch (opr)
```

```
{
```

```
// for addition operation in calculator
```

```
case 1:
```

```
cout << "You have selected the Addition Operation.";
```

```
cout << "\n Please enter the two number: \n";
```

```
cin >> num1 >> num2;
```

```
x = num1 + num2;
```

```
cout << "Sum of two number = " << x;
```

```
break;
```

```
// for subtraction operation in calculator
```

```
case 2:
```

```
cout << "You have selected the Subtraction Operation.";
```



```

cout << "\n Please enter the two number: \n";

cin >> num1 >> num2;

x = num1 - num2;

cout << "Subtraction of two number = " << x;

break;

// for multiplication operation in calculator

case 3:

cout << "You have selected the Multiplication Operation.";

cout << "\n Please enter the two number: \n";

cin >> num1 >> num2;

x = num1 * num2;

cout << "Product of two number = " << x;

break;

// for division operation in calculator

case 4:

cout << "You have selected the Division Operation.";

cout << "\n Please enter the two number; \n";

cin >> num1 >> num2;

```

```
// while loop checks for divisor whether it is zero
```

```
while ( num2 == 0)
```

```
{
```

```
cout << "\n Divisor cannot be zero"
```

```
    "\n Please enter the divisor once again: ";
```

```
    cin >> num2;
```

```
}
```

```
x = num1 / num2;
```

```
cout << "\n Quotient = " << x;
```

```
break;
```

```
// to square a number in calculator
```

```
case 5:
```

```
cout << "You have selected the Square Operation.";
```

```
cout << "\n Please enter any number: \n";
```

```
cin >> num1;
```

```
x = num1 * num1;
```

```
cout << "Square is = " << x;
```

```
break;
```

```
case 6: exit(0); // terminate the program

break;

default: cout << "\n Something went wrong..!!";

break;

}

cout << "\n----- \n";

} while(opr != 6);

getch();

}
```

Output:


```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Select an operation to perform a simple calculation in C++ Calculator
1 = Addition
2 = Subtraction
3 = Multiplication
4 = Division
5 = Square
6 = Exit

Make a choice: 3
You have selected the Multiplication Operation.
Please enter the two number:
4
3
Product of two number = 12
-----
Select an operation to perform a simple calculation in C++ Calculator
1 = Addition
2 = Subtraction
3 = Multiplication
4 = Division
5 = Square
6 = Exit

Make a choice:
```

Program to convert infix to postfix expression in C++ using the Stack

Data Structure

Infix expression

An infix expression is an expression in which operators (+, -, *, /) are written between the two operands. For example, consider the following expressions:

1. $A + B$
2. $A + B - C$
3. $(A + B) + (C - D)$

Here we have written '+' operator between the operands A and B, and the - operator in between the C and D operand.

Postfix Expression

The postfix operator also contains operator and operands. In the postfix expression, the operator is written after the operand. It is also known as **Reverse Polish Notation**. For example, consider the following expressions:

1. $AB +$
2. $AB + C -$
3. $ABC^* +$
4. $AB + C^* D -$

Algorithm to Convert Infix to Postfix Expression Using Stack

Following is the **algorithm** to convert infix expression into Reverse Polish notation.

- Initialize the Stack.
- Scan the operator from left to right in the infix expression.
- If the leftmost character is an operand, set it as the current output to the Postfix string.
- And if the scanned character is the operator and the Stack is empty or contains the '(', ')' symbol, push the operator into the Stack.

- If the scanned operator has higher precedence than the existing ~~precedence~~ operator in the Stack or if the Stack is empty, put it on the Stack.
- If the scanned operator has lower precedence than the existing operator in the Stack, pop all the Stack operators. After that, push the scanned operator into the Stack.
- If the scanned character is a left bracket '(', push it into the Stack.
- If we encountered right bracket ')', pop the Stack and print all output string character until '(' is encountered and discard both the bracket.
- Repeat all steps from 2 to 8 until the infix expression is scanned.
- Print the Stack output.
- Pop and output all characters, including the operator, from the Stack until it is not empty.

Let's translate an infix expression into postfix expression in the stack:

Here, we have infix expression $((A * (B + D) / E) - F * (G + H / K))$ to convert into its equivalent postfix expression:

Label No.	Symbol Scanned	Stack	Expression
1	(	(	
2	(	((	

3	A	((	A
4	*	((*	A
5	(	((*(	A
6	B	((*(	AB
7	+	((*(+	AB
8	D	((*(+	ABD
9	)	((*	ABD+
10	/	((*/	ABD+
11	E	((*/	ABD+E
12	)	(	ABD+E/*
13	-	(-	ABD+E/*
14	(	(-(	ABD+E/*
15	F	(-(	ABD+E/*F

16	*	(-(*	ABD+E/*F
17	(	(-(*(	ABD+E/*F
18	G	(-(*(	ABD+E/*FG
19	+	(-(*(+	ABD+E/*FG
20	H	(-(*(+	ABD+E/*FGH
21	/	(-(*(+/	ABD+E/*FGH
22	K	(-(*(+/	ABD+E/*FGHK
23	)	(-(*	ABD+E/*FGHK/+
24	)	(-	ABD+E/*FGHK/+*
25	)		ABD+E/*FGHK/+*-

Program to convert infix expression to postfix expression

Let's create a C++ program that converts infix expression to the postfix expression

```
#include<iostream>
```

```
#include<stack>
```



```
using namespace std;
```

```
// defines the Boolean function for operator, operand, equalOrhigher precedence and the string conversion function.
```

```
bool IsOperator(char);
```

```
bool IsOperand(char);
```

```
bool eqOrhigher(char, char);
```

```
string convert(string);
```

```
int main()
```

```
{
```

```
string infix_expression, postfix_expression;
```

```
int ch;
```

```
do
```

```
{
```

```
cout << " Enter an infix expression: ";
```

```
cin >> infix_expression;
```

```
postfix_expression = convert(infix_expression);
```

```
cout << "\n Your Infix expression is: " << infix_expression;
```

```
cout << "\n Postfix expression is: " << postfix_expression;
```

```
cout << "\n \t Do you want to enter infix expression (1/ 0)?";
```

```
cin >> ch;
```

```
//cin.ignore( );
```

```
} while(ch == 1);
```

```
return 0;
```

```
}
```

```
// define the IsOperator() function to validate whether any symbol is operator.
```

```
/* If the symbol is operator, it returns true, otherwise false. */
```

```
bool IsOperator(char c)
```

```
{
```

```
if(c == '+' || c == '-' || c == '*' || c == '/' || c == '^' )
```

```
return true;
```

```
return false;
```

```
}
```

```
// IsOperand() function is used to validate whether the character is operand.
```

```

bool IsOperand(char c)

{

    if( c >= 'A' && c <= 'Z') /* Define the character in between A to Z. If not, it returns False. */

        return true;

    if (c >= 'a' && c <= 'z') // Define the character in between a to z. If not, it returns False. */

        return true;

    if(c >= '0' && c <= '9') // Define the character in between 0 to 9. If not, it returns False. */

        return true;

    return false;

}

// here, precedence() function is used to define the precedence to the operator.

```

```

int precedence(char op)

{

    if(op == '+' || op == '-') /* it defines the lowest precedence */

        return 1;

```



```
if (op == '*' || op == '/')
```

```
return 2;
```

```
if(op == '^')                                /* exponent operator has the highest precedence */  
ce *
```

```
return 3;
```

```
return 0;
```

```
}
```

```
/* The eqOrhigher() function is used to check the higher or equal precedence of  
the two operators in infix expression. */
```

```
bool eqOrhigher (char op1, char op2)
```

```
{
```

```
int p1 = precedence(op1);
```

```
int p2 = precedence(op2);
```

```
if (p1 == p2)
```

```
{
```

```
if (op1 == '^')
```

```
return false;
```

```
return true;
```

```
}
```

```
return (p1>p2 ? true : false);
```

```
}
```

/* string convert() function is used to convert the infix expression to the postfix expression of the Stack */

```
string convert(string infix)
```

```
{
```

```
stack <char> S;
```

```
string postfix = "";
```

```
char ch;
```

```
S.push( '(' );
```

```
infix += ')';
```

```
for(int i = 0; i<infix.length(); i++)
```

```
{
```

```
ch = infix[i];
```

```

if(ch == ' ')

continue;

else if(ch == '(')

S.push(ch);

else if(IsOperand(ch))

postfix += ch;

else if(IsOperator(ch))

{

while(!S.empty() && eqOrhigher(S.top(), ch))

{

postfix += S.top();

S.pop();

}

S.push(ch);

}

else if(ch == ')')

{

```



```

while(!S.empty() && S.top() != '(')

{

postfix += S.top();

S.pop();

}

S.pop();

}

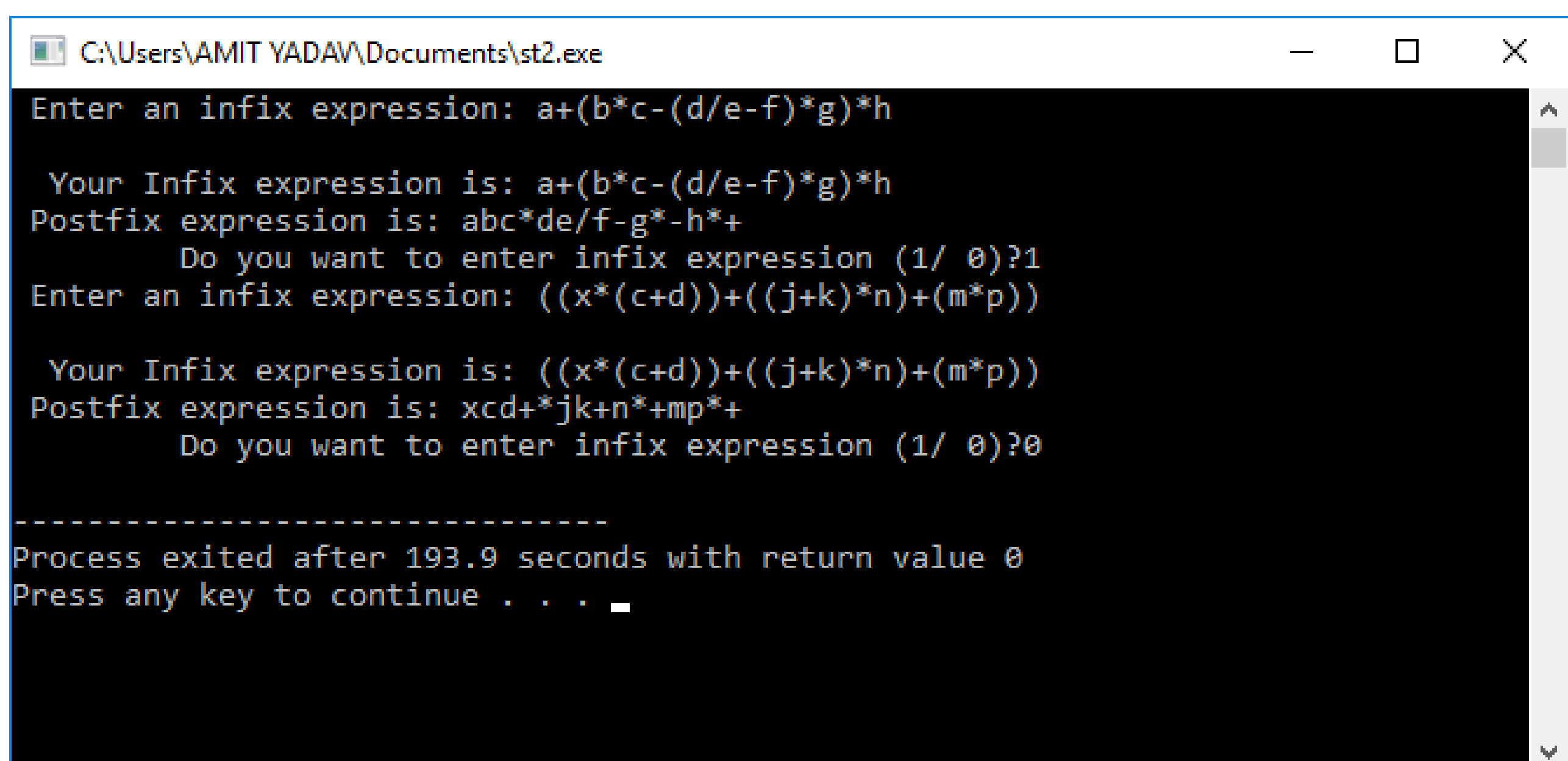
}

return postfix;

}

```

Output:



```

C:\Users\AMIT YADAV\Documents\st2.exe
Enter an infix expression: a+(b*c-(d/e-f)*g)*h
Your Infix expression is: a+(b*c-(d/e-f)*g)*h
Postfix expression is: abc*de/f-g*-h*+
Do you want to enter infix expression (1/ 0)?1
Enter an infix expression: ((x*(c+d))+((j+k)*n)+(m*p))
Your Infix expression is: ((x*(c+d))+((j+k)*n)+(m*p))
Postfix expression is: xcd+*jk+n*+mp*+
Do you want to enter infix expression (1/ 0)?0
-----
Process exited after 193.9 seconds with return value 0
Press any key to continue . . .

```

C++ program to merge two unsorted arrays

In this article, we will write a program to merge two unsorted arrays. The output is the sorted array in ascending order.

Input: a[] = {10, 5, 15}

b[] = {20, 3, 2}

Output: The merged array in sorted order {2, 3, 5, 10, 15, 20}

Input: a[] = {1, 10, 5, 15}

b[] = {20, 0, 2}

Output: The merged array in sorted order {0, 1, 2, 5, 10, 15, 20}

Approach 1

The first approach that can be used here is to concatenate both the arrays and sort the concatenated array. We create a third array of the size of the first two arrays and then transfer all the elements of both the arrays into the resultant array. After the append operation, we sort the resultant array.

C++ code

```
#include <bits/stdc++.h>

using namespace std;

void sortedMerge(int a[], int b[], int res[],
```



```

int n, int m) // Merge two arrays in unsorted manner

{

    // Concatenate two arrays

    int i = 0, j = 0, k = 0;

    while (i < n) { // iterate in first array

        res[k] = a[i]; // put each element in res array

        i += 1;

        k += 1;

    }

    while (j < m) { // iterate in the second array

        res[k] = b[j]; // put each element in res array

        j += 1;

        k += 1;

    }

    sort(res, res + n + m); // sort the res array to get the sorted Concatenate

}

int main()

{

```

```

int a[] = { 10, 5, 15, 22, 100 };

int b[] = { 20, 3, 2, 12, 1, 7 };

int n = sizeof(a) / sizeof(a[0]); // find the size of array a

int m = sizeof(b) / sizeof(b[0]); // find the size of array b

int res[n + m]; // create res array to Concatenate both the array

sortedMerge(a, b, res, n, m); // call function to append and sort

cout << "The sorted array is: ";

for (int i = 0; i < n + m; i++)

    cout << " " << res[i];

cout << "\n";

return 0;

}

```

Output

The sorted array is: 1 2 3 5 7 10 12 15 20 22 100

Time complexity

$O((n+m)\log(n+m))$, where n and m are the sizes of the arrays

Space complexity

$O(n+m)$

Approach 2

The above approach can be optimised when we use the idea of first sorting then merging it to the third array. Sort both the arrays separately and merge them into the resultant array.

C++ code

```
// CPP program to merge two unsorted lists

// in sorted order

#include <bits/stdc++.h>

using namespace std;

void sortedMerge(int a[], int b[], int res[],

    int n, int m) // Merge two arrays in unsorted manner
{
    sort(a, a + n); // Sort the array a

    sort(b, b + m); // sort the array b

    int i = 0, j = 0, k = 0;

    while (i < n && j < m) { // After the array are sorted compare and merge to t
        hird array

        if (a[i] <= b[j]) { // if element of a is less than b

            res[k] = a[i]; // put element of a into the res and increment i
```



```

        i += 1;

        k += 1;

    } else {

        res[k] = b[j]; // otherwise put the element of b into the res array and i
increment j

        j += 1;

        k += 1;

    }

}

while (i < n) { // If array a elements are left in the array put in res

    res[k] = a[i];

    i += 1;

    k += 1;

}

while (j < m) { // If array a elements are left in the array put in res

    res[k] = b[j];

    j += 1;

    k += 1;

```

```

    }

}

int main()

{

    int a[] = { 10, 5, 15, 22, 100 };

    int b[] = { 20, 3, 2, 12, 1, 7 };

    int n = sizeof(a) / sizeof(a[0]); // find the size of array a

    int m = sizeof(b) / sizeof(b[0]); // find the size of array b

    int res[n + m]; // create res array to Concatenate both the array

    sortedMerge(a, b, res, n, m); // call function to append and sort

    cout << "The sorted array is: ";

    for (int i = 0; i < n + m; i++)

        cout << " " << res[i];

    cout << "\n";

    return 0;

}

```

Output

The sorted array is: 1 2 3 5 7 10 12 15 20 22 100

Time Complexity:

$O(n \log n + m \log m + (n + m))$ // which is far better than approach 1

Space Complexity:

$O(n + m)$ It remains same as the approach 1

C++ coin change program

In this article, we will see the most asked interview problem. The coin change problem is a good example of a dynamic programming approach. Now let us understand the problem statement.

Problem statement

Given N and an array (say `coins[]`) that contains some numbers (coins in rupees). N is a coin, and the array contains various coins. The task is to make the change of N using the coins of the array. Make a change in such a way that a minimum number of coins are used.

Example

Let us take a input array `coins[] = {10, 25, 5}`, total coins = 3

We have $N = 30$.

The output is **two** as we can use one 25 rupee coin and a 5 rupee coin to make 30. ($25 + 5 = 30$)

Similarly, `coins[] = {1, 9, 6, 5}`, total coins = 4

N=13

The output is ~~three~~ as we need two 6 rupees coins and one 1 rupee coin. ($6 + 6 + 1 = 13$)

Approach 1

The approach uses a recursive method. We start with N, and in each iteration, we divide the problem into smaller subproblems. This is done by taking every coin from the coins array and decrease it from N. The case where N comes to zero gives us the required solution. For an optimal answer, we return the minimum of all combinations.

C++ code

```
#include <bits/stdc++.h>

using namespace std;

int minCoins(int coins[], int total_coins, int N) // Function to return the minimum
// coins needed
{
    if (N == 0) // If we have a combination then
        return 0;

    int result = INT_MAX; // Currently result is initialised as INT_MAX

    for (int i = 0; i < total_coins; i++) // run until availability of coins
    {
        if (coins[i] <= N) { // Add to the list of counting
```

```

        int sub_res = 1 + minCoins(coins, total_coins, N - coins[i]); // add 1 d
ue to the coin inclusion

        // see if result can minimize

        if (sub_res < result)

            result = sub_res;

    }

}

return result;

}

int main()

{

    int coins[] = { 10, 25, 5 };

    int sum = 30; // the money to convert

    int total_coins = 3; // total availability of coins

    cout << "Minmum coins needed are " << minCoins(coins, total_coins, sum);

}

```

Output

Minimum coins needed are 2

The above solution does not work for more significant inputs. In this case, the recursive approach fails.

Now let us look at an optimized approach.

Approach 2

This approach uses the idea of bottom-up dynamic programming.

Since subproblems are computed, again and again, there is a condition of overlapping subproblems. The problem of recomputation can be solved using the property of memoization, i.e., create a `dp[]` array that stores the computed values at a particular stage. At each stage calculate all the possible combinations. After all the combinations, the last value in the `dp[]` array gives us the answer.

C++ code

```
#include <bits/stdc++.h>

using namespace std;

int coins[] = { 10, 25, 5 }; // coins array

int dp[1000] = { 0 }; // array for memoisation

int minCoins(int N, int M) // N is the sum , M is total_coins
{
    for (int i = 0; i <= N; i++)

        dp[i] = INT_MAX; // Initialise all dp[] value to INT_MAX
```

```

dp[0] = 0; // Base case if sum becomes zero min rupees = 0

//Iterating in the outer loop for possible values of sum between 1 to N

//Since our final solution for sum = N might depend upon any of these values

for (int i = 1; i <= N; i++) {

    //Inner loop denotes the index of coin array.

    //For each value of sum, to obtain the optimal solution.

    for (int j = 0; j < M; j++) {

        //i > sum

        //j > next coin index

        //If we can include this coin in our solution

        if (coins[j] <= i) {

            //Solution might include the newly included coin

            dp[i] = min(dp[i], 1 + dp[i - coins[j]]);

        }

    }

}

return dp[N];

```

```

}

int main()

{

    int sum = 30; // the money to convert

    int total_coins = 3; // total availability of coins

    cout << "Minimum coins needed are " << minCoins(sum, total_coins);

}

```

Output

Minimum coins needed are 2

The time complexity of the code is $O(\text{total_coins} + \text{sum})$. This is a much-optimized approach because of storing the results at each stage and utilizing it on the other iterations.

C++ program to add two complex numbers using class

In this article, we will write a program to add two complex numbers $(a1 + ib1)$ and $(a2 + ib2)$ using class.

For example

Input: $4 + i5$ and $8 + i9$

Here $a1 = 4$ and $a2 = 8$. On adding $a1$ and $a2$, we get $(8 + 4) = 12$

Further, $b1 = 5$ and $b2 = 9$. On adding $b1$ and $b2$, we get $(5 + 9) = 14$

Output: $9 + i14$

Input: $2 + i7$ and $10 + i6$

Here $a1 = 2$ and $a2 = 10$. On adding $a1$ and $a2$, we get $(2 + 10) = 12$

Further, $b1 = 7$ and $b2 = 6$. On adding $b1$ and $b2$, we get $(7 + 6) = 13$

Output: $12 + i13$

Class construction

At first, we will create a class for complex numbers. The observation shows that there is a real number ($a1$) and an imaginary number ($b1$) in a complex number.

We need two data members to represent the complex numbers.

Below is the class structure:

```
class Complex
{
    public:
        int real; // To store real part of complex number
        int imaginary; // To store imaginary part of complex number
}
```

Constructors

Two constructors will be made to initialize the data members of the class Complex.

Non-Parameterized constructor

We need a non - parameterized constructor to initialize the default values of the data members to zero.

Below is the structure of non-parameterized constructor

```
Complex()  
  
{  
  
    real = 0;  
  
    imaginary = 0;  
  
}
```

Parameterized constructor

We need a non-parameterized constructor to initialize the data members to the value passed by the main function.

Below is the structure of parameterized constructor

```
Complex(int r, int i)  
  
{  
  
    real = r; // r is initialized during object creation  
  
    imaginary = i; // i is initialized during object creation
```

```
}
```

Algorithm

For adding two complex numbers, we will make two objects of the Complex class and initialize them with the values. After that, we will make a third object that will store the result.

C++ code

```
// C++ program to Add two complex numbers

#include <bits/stdc++.h>

using namespace std;

class Complex {

public:

    int real; // To store real part of complex number

    int imaginary; // To store imaginary part of complex number

    Complex()

    {

        // Initial values are zero

        real = 0;

        imaginary = 0;

    }

}
```



```

Complex(int r, int i)

{

    real = r; // r is initialized during object creation

    imaginary = i; // i is initialized during object creation

}

Complex addComplexNumber(Complex C1, Complex C2)

{

    Complex res; // result object of complex class

    // adding real part of complex numbers

    res.real = C1.real + C2.real;

    // adding Imaginary part of complex numbers

    res.imaginary = C1.imaginary + C2.imaginary;

    // returning the sum

    return res;

}

};

// Main Class

int main()

```

```

{

// First Complex number

Complex C1(4, 5);

// printing first complex number

cout << "Complex number 1 : " << C1.real

    << " + i" << C1.imaginary << endl;

// Second Complex number

Complex C2(8, 9);

// printing second complex number

cout << "Complex number 2 : " << C2.real

    << " + i" << C2.imaginary << endl;

// for Storing the sum

Complex C3;

// calling addComplexNumber( ) method

C3 = C3.addComplexNumber( C1, C2);

// printing the sum

cout << "Sum of complex number : "

```



```

        << C3.real << " + i"

        << C3.imaginary;

cout << endl

        << endl;

// Test for second input

// First Complex number

Complex A(2, 7);

// printing first complex number

cout << "Complex number 1 : " << A.real

        << " + i" << A.imaginary << endl;

// Second Complex number

Complex B(10, 6);

// printing second complex number

cout << "Complex number 2 : " << B.real

        << " + i" << B.imaginary << endl;

// for Storing the sum

Complex C;

// calling addComplexNumber( ) method

```

```

C = C.addComplexNumber(A, B);

// printing the sum

cout << "Sum of complex number : "

    << C.real << " + i"

    << C.imaginary;

}

```

Output

```

Complex number 1 : 4 + i5
Complex number 2 : 8 + i9
Sum of complex number : 12 + i14

Complex number 1 : 2 + i7
Complex number 2 : 10 + i6
Sum of complex number : 12 + i13

```

Time complexity

The time complexity is $O(1)$ as we have to call the function `addComplexNumber()` to add the two complex numbers.

C++ program to find the GCD of two numbers

In this tutorial, we will write a c++ program to find the GCD of two numbers.

GCD (Greatest common divisor) is also known as HCF (Highest Common Factor).

For example

$$36 = 2 * 2 * 3 * 3$$

$$60 = 2 * 2 * 3 * 5$$

The highest common factor of the two numbers is 2, 2, and 3.

So, the HCF of two numbers is $2 * 2 * 3 = 12$

$$20 = 2 * 2 * 5$$

$$28 = 2 * 2 * 7$$

The highest common factor of the two numbers is 2 and 2.

So, the HCF of two numbers is $2 * 2 = 4$.

Approach 1

The problem can be solved by finding all the prime factors of the two numbers and then find common factors and return their product.

- Find the factors of the first number
- Find the factors of the second number

- Find common factors of both numbers
- Multiply them to get GCD

C++ code

```
#include <bits/stdc++.h>
```

```
#define MAXFACTORS 1024 // Let us consider max factors can be 1024
```

```
using namespace std;
```

```
typedef struct
```

```
{
```

```
    int size;
```

```
    int factor[MAXFACTORS + 1];
```

```
    int exponent[MAXFACTORS + 1];
```

```
} FACTORIZATION;
```

```
// Function to find the factorization of M and N
```

```
void FindFactorization(int x, FACTORIZATION* factorization)
```

```
{
```

```
    int i, j = 1;
```

```
    int n = x, c = 0;
```

```

int k = 1;

factorization->factor[0] = 1;

factorization->exponent[0] = 1;

for (i = 2; i <= n; i++) {

    c = 0;

    while (n % i == 0) {

        c++;

        // factorization->factor[j]=i;

        n = n / i;

        // j++;

    }

    if (c > 0) {

        factorization->exponent[k] = c;

        factorization->factor[k] = i;

        k++;

    }

}

```

```

factorization->size = k - 1;

}

// function to find the gcd using Middle School procedure

int gcdMiddleSchoolProcedure(int m, int n)

{

    FACTORIZATION mFactorization, nFactorization;

    int r, mi, ni, i, k, x = 1, j;

    // Step 1.

    FindFactorization(m, &mFactorization);

    // Step 2.

    FindFactorization(n, &nFactorization);

    // Steps 3 and 4.

    // Procedure algorithm for computing the

    // greatest common divisor.

    int min;

    i = 1;

    j = 1;

    while (i <= mFactorization.size && j <= nFactorization.size) {

```



```

    if (mFactorization.factor[i] < nFactorization.factor[j])

        i++;

    else if (nFactorization.factor[j] < mFactorization.factor[i])

        j++;

    else /* if arr1[i] == arr2[j] */

    {

        min = mFactorization.exponent[i] > nFactorization.exponent[j]

            ? nFactorization.exponent[j]

            : mFactorization.exponent[i];

        x = x * mFactorization.factor[i] * min;

        i++;

        j++;

    }

}

return x;

}

int main()

{

```

```

int m = 36, n = 60;

cout << "GCD of " << m << " and " << n << " is "

    << gcdMiddleSchoolProcedure(m, n);

return (0);

}

```

Output

GCD of 36 and 60 is 12

Approach 2

Approach 1 can be solved in an efficient way using the Euclidean algorithm. This algorithm follows the idea that GCD does not change if the smaller number gets subtracted from the larger one.

C++ code

```

#include <iostream>

using namespace std;

int gcd(int a, int b) // The function runs recursive in nature to return GCD

{

    if (a == 0) // If a becomes zero

```



```

    return b; // b is the GCD

    if (b == 0) // If b becomes zero

        return a; // a is the GCD


    if (a == b) // The case of equal numbers

        return a; // return any one of them

    // Apply case of subtraction

    if (a > b) // if a is greater subtract b

        return gcd(a-b, b);

    return gcd(a, b-a); //otherwise subtract a
}

int main()

{

    int a = 36, b = 60;

    cout<<"GCD of "<<a<<" and "<<b<<" is "<<gcd(a, b);

    return 0;

}

```

Output

GCD of 36 and 60 is 12

Approach 3

Approach two can be optimized further to use a modulo operator instead of subtraction.

C++ code

```
#include <iostream>

using namespace std;

int gcd(int a, int b) // The function runs recursive in nature to return GCD
{
    if (b == 0) // if b becomes 0 return a
        return a;
    return gcd(b, a % b); // divide to a by b to make smaller number
}

int main()
{
    int a = 60, b = 36;

    cout<<"GCD of "<<a<<" and "<<b<<" is "<<gcd(a, b);

    return 0;
```

```
}
```

Output

GCD of 60 and 36 is 12

C++ program to find greatest of four numbers

In this tutorial, we will write a C++ program to find the greatest of four numbers.

For example

a = 10, b = 50, c = 20, d = 25

The greatest number is b 50

a = 35, b = 50, c = 99, d = 2

The greatest number is c 99

Approach 1

The approach is the traditional way of searching for the greatest among four numbers. The if condition checks whether a is greater and then use if-else to check for b, another if-else to check for c, and the last else to print d as the greatest.

Algorithm

- START
- INPUT FOUR NUMBERS A, B, C, D

- IFA > B THEN
 IFA > C THEN
 IFA > D THEN
 A IS THE GREATEST
 ELSE
 D IS THE GREATEST
- ELSE IF B > C THEN
 IF B > D THEN
 B IS THE GREATEST
 ELSE
 D IS THE GREATEST
- ELSE IF C > D THEN
 C IS THE GREATEST
- ELSE
 D IS THE GREATEST

C++ Code

```
#include <bits/stdc++.h>

using namespace std;

void find_greatest(int a, int b, int c, int d)
{
    if (a > b) {
        if (a > c) {
```

```
    if (a > d) {  
  
        cout << "a is greatest";  
  
    }  
  
    else {  
  
        cout << "d is greatest";  
  
    }  
  
}  
  
else if (b > c) {  
  
    if (b > d) {  
  
        cout << "b is greatest";  
  
    }  
  
    else {  
  
        cout << "d is greatest";  
  
    }  
  
}  
  
else if (c > d) {  
  
    cout << "c is greatest";  
  
}
```

```

    }

    else {

        cout << "d is greatest";

    }

}

int main()

{

    int a = 10, b = 50, c = 20, d = 25;

    cout << "a=" << a << " b=" << b << " c=" << c << " d=" << d;

    cout << "\n";

    find_greatest(a, b, c, d);

    a = 35, b = 50, c = 99, d = 2;

    cout << "\n";

    cout << "a=" << a << " b=" << b << " c=" << c << " d=" << d;

    cout << "\n";

    find_greatest(a, b, c, d);

    return 0;

}

```


Output

```
a=10 b=50 c=20 d=25
```

```
b is greatest
```

```
a=35 b=50 c=99 d=2
```

```
c is greatest
```

Approach 2

This approach uses the inbuilt max function.

Here is the syntax of max function

```
template<constexpr const T&max(const T&a, const T&b);
```

Here, a and b are the numbers to be compared.

Return: Larger of the two values.

For example

`std::max(2,5)` will return 5

So, to find out the maximum of 4 numbers, we can use chaining of a max function as follows -

```
int x = max(a, max(b, max(c, d)));
```

C++ code

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void find_greatest(int a, int b, int c, int d)
```

```
{
```

```
    int x = max(a, max(b, max(c, d)));
```

```
    if (x == a)
```

```
        cout << "a is greatest";
```

```
    if (x == b)
```

```
        cout << "b is greatest";
```

```
    if (x == c)
```

```
        cout << "c is greatest";
```

```
    if (x == d)
```

```
        cout << "d is greatest";
```

```
}
```

```
int main()
```

```
{
```

```
    int a = 10, b = 50, c = 20, d = 25;
```

```
    cout << "a=" << a << " b=" << b << " c=" << c << " d=" << d;
```

```
    cout << "\n";
```



```
find_greatest(a, b, c, d);

a = 35, b = 50, c = 99, d = 2;

cout << "\n";

cout << "a=" << a << " b=" << b << " c=" << c << " d=" << d;

cout << "\n";

find_greatest(a, b, c, d);

return 0;

}
```

Output

a=10 b=50 c=20 d=25

b is greatest

a=35 b=50 c=99 d=2

c is greatest